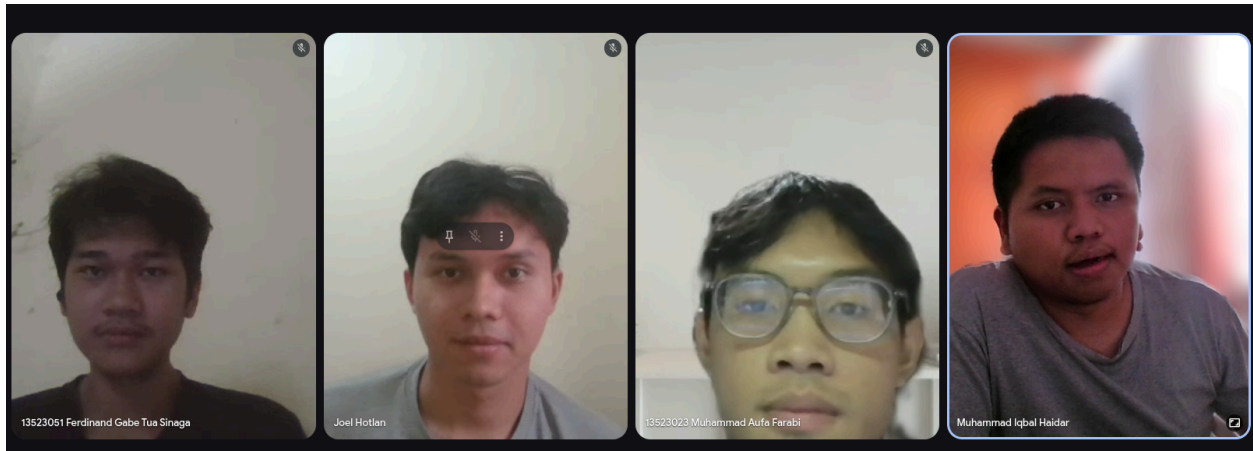


IF2224 TEORI BAHASA FORMAL DAN OTOMATA SEMESTER II TAHUN 2025/2026

PASCAL-S COMPILER



Milestone 3 - Semantic Analysis

Kelompok LPH

Muhammad Aufa Farabi	13523023
Joel Hotlan Haris Siahaan	13523025
Ferdinand Gabe Tua Sinaga	13523051
Muhammad Iqbal Haidar	13523111

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2025**

DAFTAR ISI

DAFTAR ISI.....	2
LANDASAN TEORI.....	3
PERANCANGAN & IMPLEMENTASI.....	4
PENGUJIAN.....	8
Kasus 1.....	8
Kasus 2.....	8
Kasus 3.....	8
Kasus 4.....	8
Kasus 5.....	8
KESIMPULAN DAN SARAN.....	9
LAMPIRAN.....	9
REFERENSI.....	9

LANDASAN TEORI

Pada tahapan semantic analysis ini, dilakukan pengecekan pada parse tree untuk memastikan bahwa makna dari program benar. Semantic analysis terbagi menjadi dua proses utama yakni mengubah parse tree menjadi bentuk abstract syntax tree dan type scope checking.

Proses perubahan menjadi abstract syntax tree didasari oleh banyaknya node yang merupakan node tidak esensial sehingga membuat parse tree rumit untuk dilakukan type & scope checking. Abstract syntax tree mencoba untuk menghilangkan node yang tidak esensial tersebut tanpa mengubah maksud dari parse tree sebelumnya. Pada prosesnya, digunakan syntax directed translation yakni menyisipkan semantic action ke dalam grammar. Sehingga ketika node grammar dikunjungi maka semantic action akan dieksekusi dan melakukan pembuatan node AST baru. Hasil akhir dari proses ini adalah sebuah tree yang lebih sederhana sehingga mudah untuk diproses ke bagian selanjutnya.

Selanjutnya adalah type & scope checking, tujuannya adalah pengecekan abstract syntax tree untuk memastikan tipe data dan scope dari identifier valid. Pada prosesnya digunakan symbol table tab, atab, dan btab untuk manajemen tipe dan scope dari tiap identifier. Pengecekan dilakukan secara DFS kiri mengunjungi setiap node pada abstract syntax tree menggunakan visit function. Setiap visit function berkorespondensi satu-satu dengan jenis node AST, fungsi inilah yang melakukan type & scope checking pada abstract syntax tree. Hasil dari proses ini adalah simbol table yang telah terisi dan abstract syntax tree dengan node yang sudah didekorasi dengan beberapa atribut pada symbol table tab

PERANCANGAN & IMPLEMENTASI

Untuk mengimplementasikan semantic analysis pada milestone kali ini, kelompok kami memutuskan untuk membuat tiga kelas utama, yaitu kelas *ASTBuilder*, *ASTPrinter*, dan *SemanticAnalyzer*. Selain itu kami juga membuat `__Node` untuk merepresentasikan setiap node pada abstract syntax tree.

Kelas *ASTBuilder* bertugas untuk mengubah parse tree hasil parser menjadi sebuah abstract syntax tree. Pembangunan abstract syntax tree dilakukan dengan mengunjungi setiap node pada abstract tree lalu melakukan semantic action yang sesuai dengan grammar/node tersebut

Implementasi: Class <i>ASTBuilder</i>	
Fungsi	Keterangan
<code>__init__(root: TreeNode):</code>	Fungsi ini merupakan constructor dari kelas <i>ASTBuilder</i> . Melakukan set atribut root berupa parse tree hasil parser
<code>build()</code>	Fungsi ini digunakan untuk melakukan pembangunan AST dengan atribut root sebagai parse treenya
<code>visit(node: TreeNode, raw_str: string)</code>	Fungsi ini merupakan implementasi visit function dengan visitor pattern
<code>_clean(name: string):</code>	Fungsi ini digunakan untuk merubah nama node pada parse tree menjadi nama yang sesuai dengan visit function
<code>_extract_value(raw: string)</code>	Fungsi ini digunakan untuk mengekstraksi nilai pada string token hasil lexer
<code>generic_visit(node)</code>	Fungsi ini merupakan fallback apabila terdapat nama node yang tidak ada implementasi visit functionnya
<code>visit_{parse-tree-node-name}</code>	Fungsi ini merupakan visit function untuk setiap node pada parse tree. Fungsi ini dieksekusi setiap kali mengunjungi node parse tree dengan nama yang sama. Pada fungsi ini dilakukan eksekusi semantic action sesuai dengan grammar yang sudah ditentukan.

Kelas `__Node` merupakan representasi dari setiap node pada abstract syntax tree. Seluruh kelas `__Node` memiliki atribut `id`, `type`, `lev` yang berguna untuk menyimpan data esensial dari symbol table untuk node tersebut dan juga untuk memudahkan display decorated AST. beberapa contoh kelas antara lain

Implementasi: Class __Node	
Kelas	Keterangan
ProgramNode	Merepresentasikan node paling atas pada AST. Memiliki atribut name, declarations, block
BlockNode	Merepresentasikan node block pada AST. Memiliki atribut id, type, lev
ConstDeclNode	Merepresentasikan node deklarasi konstanta pada AST. Memiliki atribut name, value
TypeNode	Merepresentasikan node deklarasi tipe pada AST. Memiliki atribut name, type_node
AssignNode	Merepresentasikan node assignment pada AST. Memiliki atribut target, value
BinOpNode	Merepresentasikan node binary operation (relational, arithmetic, logical). Memiliki atribut op, left, right.

Kelas ASTPrinter bertugas untuk mendisplay AST yang telah dibangun sebelumnya sesuai dengan format yang ditentukan pada spesifikasi. ASTPrinter ini bisa menerima AST biasa maupun yang sudah didekorasi.

Implementasi: Class ASTPrinter	
Fungsi	Keterangan
print(node, indent: integer)	Fungsi ini digunakan untuk print AST sesuai format pada spesifikasi, dengan root node AST nya adalah parameter node. Fungsi ini mengimplementasikan visitor pattern
_indent(level: integer)	Fungsi ini merupakan helper indentasi untuk mengenerate string kosong sepanjang level
_get_decoration(node)	Fungsi ini merupakan helper untuk mengenerate string gabungan yang berisi atribut symbol table pada node yakni id, type, lev.
_print_header(node, label: string, indent: integer)	Fungsi ini merupakan helper untuk mencetak baris judul node beserta dekorasinya

<code>_print_section(label, content, indent)</code>	Fungsi ini merupakan helper untuk mencetak labeled section
<code>_print_labeled_value(label, value, indent)</code>	Fungsi ini merupakan helper untuk mencetak labeled value
<code>print_generic(node, indent)</code>	Fungsi ini merupakan fallback ketika menemukan sebuah node AST yang belum didefinisikan fungsi print nya
<code>print_{AST-Node-Class-Name}</code>	Fungsi ini digunakan untuk display node pada AST. Nama fungsi berkorespondensi satu-satu dengan nama kelas node AST

Kelas SemanticAnalyzer bertugas untuk melakukan proses type & scope checking. Fungsi ini mengunjungi setiap node pada AST untuk memastikan bahwa makna dari node tersebut valid. Pengecekan yang dilakukan adalah memastikan tipe data sesuai dengan ketentuan dan memastikan scope identifier sesuai dengan yang ada pada symbol table.

Implementasi: Class SemanticAnalyzer	
Fungsi	Keterangan
<code>__init__(self)</code>	Fungsi ini merupakan constructor dari kelas SemanticAnalyzer. Pada fungsi ini dilakukan inisiasi symbol table (termasuk memasukkan 29 reserved words ke tab, dan global scope ke btab)
<code>visit(node)</code>	Fungsi ini merupakan implementasi visit function dengan visitor pattern
<code>generic_visit(node)</code>	Fungsi ini merupakan fallback apabila terdapat node yang tidak ada implementasi visit functionnya
<code>decorate(node, type, idx, lev)</code>	Fungsi ini merupakan helper untuk mengisi nilai atribut symbol table pada node
<code>lookup(name)</code>	Fungsi ini merupakan implementasi pencarian identifier pada symbol table tab. Pencarian dilakukan dari level/scope saat ini, lalu mundur sampai ke level/scope global
<code>lookup_block(name, btab_idx)</code>	Fungsi ini merupakan implementasi pencarian identifier pada symbol table tab, namun dibatasi hanya pada level/scope/block saat ini.

<code>enter_block()</code>	Fungsi ini merupakan helper ketika memasuki block baru. Fungsi ini melakukan penambahan entry btab, increment lev, dan pembaruan variable display[level]
<code>exit_block()</code>	Fungsi ini merupakan helper ketika keluar dari block. Fungsi ini melakukan decrement lev
<code>get_type_size(type_code, type_ref)</code>	Fungsi ini digunakan untuk mendapatkan ukuran (size) dari entry
<code>insert_tab(name, obj, type_code, ref, nrm, adr, init)</code>	Fungsi ini merupakan helper untuk memasukkan entry baru ke dalam table symbol tab
<code>insert_atab(xtyp, etyp, low, high, eref)</code>	Fungsi ini merupakan helper untuk memasukkan entry baru ke dalam table symbol atab
<code>visit_ProgramNode(node)</code>	Fungsi ini melakukan type & scope checking untuk node AST ProgramNode
<code>visit_BlockNode(node)</code>	Fungsi ini melakukan type & scope checking untuk node AST BlockNode
<code>visit_ConstDeclNode(node)</code>	Fungsi ini melakukan type & scope checking untuk node AST ConstDeclNode
<code>visit_{AST-Node-Class-Name}</code>	Fungsi ini digunakan untuk melakukan type & scope checking pada node AST. Nama fungsi berkorespondensi satu-satu dengan nama kelas node AST

PENGUJIAN

Kasus 1

Input file: test1.pas

```
program HitungLuas;

konstanta
  PI = 3.14;
  booleanconst = true;

variabel
  x: boolean;
  r: integer;
  luas: real;

mulai
  x := false;
  r := 10;
  luas := PI * r * r;
selesai.
```

Output

```
===== AST Output =====
Program: HitungLuas
Declarations:
  ConstDecl: PI
    Value:
      Number: 3.14
  ConstDecl: booleanconst
    Value:
      Boolean: True
  VarDecl: x
    Type:
      BuiltinType: boolean
  VarDecl: r
    Type:
      BuiltinType: integer
  VarDecl: luas
    Type:
      BuiltinType: real
Body:
  Block:
    Assignment:
      Target:
        Variable: x
      Value:
        Boolean: False
    Assignment:
      Target:
        Variable: r
      Value:
        Number: 10
    Assignment:
```



```

Target:
Variable: luas
Value:
  BinOp: *
    Left:
      BinOp: *
        Left:
          Variable: PI
        Right:
          Variable: r
      Right:
        Variable: r

```

===== Semantic Analysis Output =====

=====

TAB (Identifier Table)

=====

idx	identifiers	link	obj	type	ref	nrm	lev
adr	init						

0	dan	-1	[6] keyword	[0] undef	-1	0	-1
0	1						
1	larik	0	[6] keyword	[0] undef	-1	0	-1
0	1						
2	mulai	1	[6] keyword	[0] undef	-1	0	-1
0	1						
3	kasus	2	[6] keyword	[0] undef	-1	0	-1
0	1						
4	konstanta	3	[6] keyword	[0] undef	-1	0	-1
0	1						
5	div	4	[6] keyword	[0] undef	-1	0	-1
0	1						
6	turunke	5	[6] keyword	[0] undef	-1	0	-1
0	1						
7	lakukan	6	[6] keyword	[0] undef	-1	0	-1
0	1						
8	selainitu	7	[6] keyword	[0] undef	-1	0	-1
0	1						
9	selesai	8	[6] keyword	[0] undef	-1	0	-1
0	1						
10	untuk	9	[6] keyword	[0] undef	-1	0	-1
0	1						
11	fungsi	10	[6] keyword	[0] undef	-1	0	-1
0	1						
12	jika	11	[6] keyword	[0] undef	-1	0	-1
0	1						
13	mod	12	[6] keyword	[0] undef	-1	0	-1
0	1						
14	tidak	13	[6] keyword	[0] undef	-1	0	-1
0	1						
15	dari	14	[6] keyword	[0] undef	-1	0	-1
0	1						
16	atau	15	[6] keyword	[0] undef	-1	0	-1

0	1						
17	prosedur	16	[6] keyword	[0] undef	-1	0	-1
0	1						
18	program	17	[6] keyword	[0] undef	-1	0	-1
0	1						
19	rekaman	18	[6] keyword	[0] undef	-1	0	-1
0	1						
20	ulangi	19	[6] keyword	[0] undef	-1	0	-1
0	1						
21	string	20	[6] keyword	[0] undef	-1	0	-1
0	1						
22	maka	21	[6] keyword	[0] undef	-1	0	-1
0	1						
23	ke	22	[6] keyword	[0] undef	-1	0	-1
0	1						
24	tipe	23	[6] keyword	[0] undef	-1	0	-1
0	1						
25	sampai	24	[6] keyword	[0] undef	-1	0	-1
0	1						
26	var	25	[6] keyword	[0] undef	-1	0	-1
0	1						
27	selama	26	[6] keyword	[0] undef	-1	0	-1
0	1						
28	packed	27	[6] keyword	[0] undef	-1	0	-1
0	1						
29	HitungLuas	-1	[5] program	[0] undef	-1	1	0
0	1						
30	PI	29	[0] konstanta	[2] real	-1	1	0
3	1						
31	booleanconst	30	[0] konstanta	[3] boolean	-1	1	0
1	1						
32	x	31	[1] variabel	[3] boolean	-1	1	0
5	1						
33	r	32	[1] variabel	[1] int	-1	1	0
6	1						
34	luas	33	[1] variabel	[2] real	-1	1	0
7	1						

BTAB (Block Table)

blocks	last	lpar	psze	vsze
0	28	0	0	0
1	34	-1	0	3

ATAB (Array Table)

(Empty)

Decorated AST Output

```

Program: HitungLuas
Declarations:
  ConstDecl: PI  -> index:30, type:real, lev:0
    Value:
      Number: 3.14  -> type:real
  ConstDecl: booleanconst  -> index:31, type:boolean, lev:0
    Value:
      Boolean: True  -> type:boolean
  VarDecl: x  -> index:32, type:boolean, lev:0
    Type:
      BuiltinType: boolean  -> type:boolean
  VarDecl: r  -> index:33, type:integer, lev:0
    Type:
      BuiltinType: integer  -> type:integer
  VarDecl: luas  -> index:34, type:real, lev:0
    Type:
      BuiltinType: real  -> type:real
Body:
  Block:  -> index:1, lev:0
    Assignment:  -> type:void
      Target:
        Variable: x  -> index:32, type:boolean, lev:0
      Value:
        Boolean: False  -> type:boolean
    Assignment:  -> type:void
      Target:
        Variable: r  -> index:33, type:integer, lev:0
      Value:
        Number: 10  -> type:integer
    Assignment:  -> type:void
      Target:
        Variable: luas  -> index:34, type:real, lev:0
      Value:
        BinOp: *  -> type:real
          Left:
            BinOp: *  -> type:real
              Left:
                Variable: PI  -> index:30, type:real
              Right:
                Variable: r  -> index:33, type:integer
          Right:
            Variable: r  -> index:33, type:integer

```

Kasus 2

Input file: test2.pas

```

program CekScope;

variabel
  globalVar: integer;

prosedur hitung(param: integer);
variabel
  lokalVar: integer;

```

```

mulai
    lokalVar := 50;
    globalVar := param + lokalVar;
selesai;

mulai
    globalVar := 10;
    hitung(globalVar);
selesai.

```

Output

```

===== AST Output =====
Program: CekScope
Declarations:
  VarDecl: globalVar
    Type:
      BuiltinType: integer
  Procedure: hitung
    Parameters:
      param:
        BuiltinType: integer
    Declarations:
      VarDecl: lokalVar
        Type:
          BuiltinType: integer
    Body:
      Block:
        Assignment:
          Target:
            Variable: lokalVar
          Value:
            Number: 50
        Assignment:
          Target:
            Variable: globalVar
          Value:
            BinOp: +
              Left:
                Variable: param
              Right:
                Variable: lokalVar
      Body:
        Block:
          Assignment:
            Target:
              Variable: globalVar
            Value:
              Number: 10
          ProcedureCall: hitung
            Arguments:
              Variable: globalVar
===== Semantic Analysis Output =====
=====

```



```

    Target:
      Variable: lokalVar -> index:33, type:integer, lev:1
    Value:
      Number: 50 -> type:integer
  Assignment: -> type:void
  Target:
    Variable: globalVar -> index:30, type:integer, lev:0
  Value:
    BinOp: + -> type:integer
    Left:
      Variable: param -> index:32, type:integer
    Right:
      Variable: lokalVar -> index:33, type:integer
Body:
  Block: -> index:1, lev:0
  Assignment: -> type:void
  Target:
    Variable: globalVar -> index:30, type:integer, lev:0
  Value:
    Number: 10 -> type:integer
  ProcedureCall: hitung -> index:31, lev:0
  Arguments:
    Variable: globalVar -> index:30, type:integer

```

Kasus 3

Input file: test3.pas

```

program DataSiswa;

tipe
  Siswa = rekaman
    id: integer;
    nilai: real;
  selesai;

variabel
  kelas: larik [1..30] dari Siswa;
  temp: Siswa;

mulai
  temp.id := 101;
  temp.nilai := 85.5;
  kelas[1] := temp;
selesai.

```

Output

```

===== AST Output =====
Program: DataSiswa
  Declarations:
    TypeDecl: Siswa
    Definition:
      RecordType:

```


3	kasus	2	[6] keyword	[0] undef	-1	0	-1	0
1								
4	konstanta	3	[6] keyword	[0] undef	-1	0	-1	0
1								
5	div	4	[6] keyword	[0] undef	-1	0	-1	0
1								
6	turunke	5	[6] keyword	[0] undef	-1	0	-1	0
1								
7	lakukan	6	[6] keyword	[0] undef	-1	0	-1	0
1								
8	selainitu	7	[6] keyword	[0] undef	-1	0	-1	0
1								
9	selesai	8	[6] keyword	[0] undef	-1	0	-1	0
1								
10	untuk	9	[6] keyword	[0] undef	-1	0	-1	0
1								
11	fungsi	10	[6] keyword	[0] undef	-1	0	-1	0
1								
12	jika	11	[6] keyword	[0] undef	-1	0	-1	0
1								
13	mod	12	[6] keyword	[0] undef	-1	0	-1	0
1								
14	tidak	13	[6] keyword	[0] undef	-1	0	-1	0
1								
15	dari	14	[6] keyword	[0] undef	-1	0	-1	0
1								
16	atau	15	[6] keyword	[0] undef	-1	0	-1	0
1								
17	prosedur	16	[6] keyword	[0] undef	-1	0	-1	0
1								
18	program	17	[6] keyword	[0] undef	-1	0	-1	0
1								
19	rekaman	18	[6] keyword	[0] undef	-1	0	-1	0
1								
20	ulangi	19	[6] keyword	[0] undef	-1	0	-1	0
1								
21	string	20	[6] keyword	[0] undef	-1	0	-1	0
1								
22	maka	21	[6] keyword	[0] undef	-1	0	-1	0
1								
23	ke	22	[6] keyword	[0] undef	-1	0	-1	0
1								
24	tipe	23	[6] keyword	[0] undef	-1	0	-1	0
1								
25	sampai	24	[6] keyword	[0] undef	-1	0	-1	0
1								
26	var	25	[6] keyword	[0] undef	-1	0	-1	0
1								
27	selama	26	[6] keyword	[0] undef	-1	0	-1	0
1								
28	packed	27	[6] keyword	[0] undef	-1	0	-1	0
1								
29	DataSiswa	-1	[5] program	[0] undef	-1	1	0	0
1								
30	id	-1	[1] variabel	[1] int	-1	1	1	0
1								

```

31      nilai          30      [1] variabel    [2] real      -1      1      1      1
1
32      Siswa          29      [2] tipe      [6] record     2      1      0      2
1
33      kelas          32      [1] variabel    [5] array      0      1      0      5
1
34      temp           33      [1] variabel    [6] record     2      1      0
65      1

```

```

=====
BTAB (Block Table)
=====
blocks      last      lpar      psze      vsze
-----
0           28        0         0         0
1           34       -1         0         62
2           31        0         0         2

```

```

=====
==
ATAB (Array Table)
=====
==
arrays      xtyp      etyp      eref      low      high      elsz      size
-----
--
0           [1] int    [6] record 2         1        30        2        60
=====
==

```

```

===== Decorated AST Output =====
Program: DataSiswa
Declarations:
  TypeDecl: Siswa -> index:32, type:rekaman, lev:0
  Definition:
    RecordType: -> index:2, type:rekaman
    id:
      BuiltinType: integer -> type:integer
    nilai:
      BuiltinType: real -> type:real
  VarDecl: kelas -> index:33, type:larik, lev:0
  Type:
    ArrayType: [1..30] -> index:0, type:larik
    ElementType:
      CustomType: Siswa -> index:32, type:rekaman, lev:0
  VarDecl: temp -> index:34, type:rekaman, lev:0
  Type:
    CustomType: Siswa -> index:32, type:rekaman, lev:0
Body:
  Block: -> index:1, lev:0
  Assignment: -> type:void
  Target:
    RecordAccess: -> index:2, lev:30
    Variable:
      Variable: temp -> index:34, type:rekaman
      Field: id

```

```

    Value:
      Number: 101 -> type:integer
    Assignment: -> type:void
    Target:
      RecordAccess: -> index:2, lev:31
      Variable:
        Variable: temp -> index:34, type:rekaman
        Field: nilai
    Value:
      Number: 85.5 -> type:real
    Assignment: -> type:void
    Target:
      ArrayAccess: -> index:0
      Variable:
        Variable: kelas -> index:33, type:larik
      Index:
        Number: 1 -> type:integer
    Value:
      Variable: temp -> index:34, type:rekaman

```

Kasus 4

Input file: test4.pas

```

program LogikaIndo;

variabel
  a, b: integer;
  hasil: real;
  isValid: boolean;

mulai
  a := 20;
  b := 5;

  isValid := (a > 10) dan (b < 10);

  jika isValid maka
    hasil := a bagi b;

  selama a > 0 lakukan
    a := a - 1;
selesai.

```

Output

```

===== AST Output =====
Program: LogikaIndo
Declarations:
  VarDecl: a
  Type:
    BuiltinType: integer
  VarDecl: b
  Type:
    BuiltinType: integer

```

```
VarDecl: hasil
  Type:
    BuiltinType: real
VarDecl: isValid
  Type:
    BuiltinType: boolean
Body:
  Block:
    Assignment:
      Target:
        Variable: a
      Value:
        Number: 20
    Assignment:
      Target:
        Variable: b
      Value:
        Number: 5
    Assignment:
      Target:
        Variable: isValid
      Value:
        BinOp: dan
        Left:
          BinOp: >
          Left:
            Variable: a
          Right:
            Number: 10
        Right:
          BinOp: <
          Left:
            Variable: b
          Right:
            Number: 10
    If:
      Condition:
        Variable: isValid
      Then:
        Assignment:
          Target:
            Variable: hasil
          Value:
            BinOp: bagi
            Left:
              Variable: a
            Right:
              Variable: b
    While:
      Condition:
        BinOp: >
        Left:
          Variable: a
        Right:
          Number: 0
      Body:
```

```

Assignment:
Target:
Variable: a
Value:
BinOp: -
Left:
Variable: a
Right:
Number: 1

```

===== Semantic Analysis Output =====

=====

TAB (Identifier Table)

=====

idx	identifiers	link	obj	type	ref	nrm	lev
adr	init						

0	dan	-1	[6] keyword	[0] undef	-1	0	-1 0
1							
1	larik	0	[6] keyword	[0] undef	-1	0	-1 0
1							
2	mulai	1	[6] keyword	[0] undef	-1	0	-1 0
1							
3	kasus	2	[6] keyword	[0] undef	-1	0	-1 0
1							
4	konstanta	3	[6] keyword	[0] undef	-1	0	-1 0
1							
5	div	4	[6] keyword	[0] undef	-1	0	-1 0
1							
6	turunke	5	[6] keyword	[0] undef	-1	0	-1 0
1							
7	lakukan	6	[6] keyword	[0] undef	-1	0	-1 0
1							
8	selainitu	7	[6] keyword	[0] undef	-1	0	-1 0
1							
9	selesai	8	[6] keyword	[0] undef	-1	0	-1 0
1							
10	untuk	9	[6] keyword	[0] undef	-1	0	-1 0
1							
11	fungsi	10	[6] keyword	[0] undef	-1	0	-1 0
1							
12	jika	11	[6] keyword	[0] undef	-1	0	-1 0
1							
13	mod	12	[6] keyword	[0] undef	-1	0	-1 0
1							
14	tidak	13	[6] keyword	[0] undef	-1	0	-1 0
1							
15	dari	14	[6] keyword	[0] undef	-1	0	-1 0
1							
16	atau	15	[6] keyword	[0] undef	-1	0	-1 0
1							
17	prosedur	16	[6] keyword	[0] undef	-1	0	-1 0
1							


```

VarDecl: b -> index:31, type:integer, lev:0
  Type:
    BuiltinType: integer -> type:integer
VarDecl: hasil -> index:32, type:real, lev:0
  Type:
    BuiltinType: real -> type:real
VarDecl: isValid -> index:33, type:boolean, lev:0
  Type:
    BuiltinType: boolean -> type:boolean
Body:
  Block: -> index:1, lev:0
    Assignment: -> type:void
      Target:
        Variable: a -> index:30, type:integer, lev:0
      Value:
        Number: 20 -> type:integer
    Assignment: -> type:void
      Target:
        Variable: b -> index:31, type:integer, lev:0
      Value:
        Number: 5 -> type:integer
    Assignment: -> type:void
      Target:
        Variable: isValid -> index:33, type:boolean, lev:0
      Value:
        BinOp: dan -> type:boolean
          Left:
            BinOp: > -> type:boolean
              Left:
                Variable: a -> index:30, type:integer
              Right:
                Number: 10 -> type:integer
          Right:
            BinOp: < -> type:boolean
              Left:
                Variable: b -> index:31, type:integer
              Right:
                Number: 10 -> type:integer
    If: -> lev:0
      Condition:
        Variable: isValid -> index:33, type:boolean
      Then:
        Assignment: -> type:void
          Target:
            Variable: hasil -> index:32, type:real, lev:0
          Value:
            BinOp: bagi -> type:integer
              Left:
                Variable: a -> index:30, type:integer
              Right:
                Variable: b -> index:31, type:integer
    While: -> lev:0
      Condition:
        BinOp: > -> type:boolean
          Left:
            Variable: a -> index:30, type:integer

```

```

    Right:
      Number: 0  -> type:integer
  Body:
    Assignment:  -> type:void
    Target:
      Variable: a  -> index:30, type:integer, lev:0
    Value:
      BinOp: -  -> type:integer
      Left:
        Variable: a  -> index:30, type:integer
      Right:
        Number: 1  -> type:integer

```

Kasus 5

Input file: test5_error.pas

```

program TestError;

variabel
  angka: integer;
  teks: char;

mulai
  angka := 100;
  teks := 'A';

  angka := angka + teks;
selesai.

```

Output

```

===== AST Output =====
Program: TestError
Declarations:
  VarDecl: angka
    Type:
      BuiltinType: integer
  VarDecl: teks
    Type:
      BuiltinType: char
Body:
  Block:
    Assignment:
      Target:
        Variable: angka
      Value:
        Number: 100
    Assignment:
      Target:
        Variable: teks
      Value:
        Char: 'A'
    Assignment:
      Target:

```



```
    Variable: angka
Value:
    BinOp: +
    Left:
        Variable: angka
    Right:
        Variable: teks
```

[SEMANTIC ERROR]: Invalid operand types for '+'

Kasus 6

Input file: test6.pas

```
program CekScope;

tipe
  Bil = integer;
  Mahasiswa = rekaman
    nama : char;
    umur : integer;
  selesai;

variabel
  globalVar: integer;

prosedur hitung(param: integer);
variabel
  lokalVar: integer;
mulai
  lokalVar := 50;
  globalVar := param + lokalVar;
selesai;

mulai
  globalVar := 10;
  hitung(globalVar);
selesai.
```

Output

```
===== AST Output =====
Program: CekScope
Declarations:
  TypeDecl: Bil
  Definition:
    BuiltinType: integer
  TypeDecl: Mahasiswa
  Definition:
    RecordType:
      nama:
        BuiltinType: char
      umur:
        BuiltinType: integer
```


4	konstanta	3	[6] keyword	[0] undef	-1	0	-1	0
1								
5	div	4	[6] keyword	[0] undef	-1	0	-1	0
1								
6	turunke	5	[6] keyword	[0] undef	-1	0	-1	0
1								
7	lakukan	6	[6] keyword	[0] undef	-1	0	-1	0
1								
8	selainitu	7	[6] keyword	[0] undef	-1	0	-1	0
1								
9	selesai	8	[6] keyword	[0] undef	-1	0	-1	0
1								
10	untuk	9	[6] keyword	[0] undef	-1	0	-1	0
1								
11	fungsi	10	[6] keyword	[0] undef	-1	0	-1	0
1								
12	jika	11	[6] keyword	[0] undef	-1	0	-1	0
1								
13	mod	12	[6] keyword	[0] undef	-1	0	-1	0
1								
14	tidak	13	[6] keyword	[0] undef	-1	0	-1	0
1								
15	dari	14	[6] keyword	[0] undef	-1	0	-1	0
1								
16	atau	15	[6] keyword	[0] undef	-1	0	-1	0
1								
17	prosedur	16	[6] keyword	[0] undef	-1	0	-1	0
1								
18	program	17	[6] keyword	[0] undef	-1	0	-1	0
1								
19	rekaman	18	[6] keyword	[0] undef	-1	0	-1	0
1								
20	ulangi	19	[6] keyword	[0] undef	-1	0	-1	0
1								
21	string	20	[6] keyword	[0] undef	-1	0	-1	0
1								
22	maka	21	[6] keyword	[0] undef	-1	0	-1	0
1								
23	ke	22	[6] keyword	[0] undef	-1	0	-1	0
1								
24	tipe	23	[6] keyword	[0] undef	-1	0	-1	0
1								
25	sampai	24	[6] keyword	[0] undef	-1	0	-1	0
1								
26	var	25	[6] keyword	[0] undef	-1	0	-1	0
1								
27	selama	26	[6] keyword	[0] undef	-1	0	-1	0
1								
28	packed	27	[6] keyword	[0] undef	-1	0	-1	0
1								
29	CekScope	-1	[5] program	[0] undef	-1	1	0	0
1								
30	Bil	29	[2] tipe	[1] int	-1	1	0	1
1								
31	nama	-1	[1] variabel	[4] char	-1	1	1	0
1								


```

Body:
  Block: -> index:3, lev:1
    Assignment: -> type:void
      Target:
        Variable: lokalVar -> index:37, type:integer, lev:1
      Value:
        Number: 50 -> type:integer
    Assignment: -> type:void
      Target:
        Variable: globalVar -> index:34, type:integer, lev:0
      Value:
        BinOp: + -> type:integer
          Left:
            Variable: param -> index:36, type:integer
          Right:
            Variable: lokalVar -> index:37, type:integer
Body:
  Block: -> index:1, lev:0
    Assignment: -> type:void
      Target:
        Variable: globalVar -> index:34, type:integer, lev:0
      Value:
        Number: 10 -> type:integer
    ProcedureCall: hitung -> index:35, lev:0
      Arguments:
        Variable: globalVar -> index:34, type:integer

```

Kasus 7

Input file: test7.pas

```

program TestSemantic;

konstanta
  MAX = 10;
  PI = 3.14;
  NAME = 'Pascals';

tipe
  Vector = larik [1..10] dari integer;

  Point = rekaman
    x, y: integer;
  selesai;

  Shape = rekaman
    id: integer;
    vertices: Vector;
    center: Point;
    active: boolean;
  selesai;

variabel
  gval, i: integer;
  greal: real;
  isvalid: boolean;

```

```

myshape: Shape;
coords: larik [1..5] dari Point;
writeln: integer;

fungsi factorial(n: integer): integer;
mulai
    jika n <= 1 maka
        factorial := 1
    selainitu
        factorial := n * factorial(n - 1);
selesai;

prosedur swap(var a, b: integer);
variabel
    temp: integer;
mulai
    temp := a;
    a := b;
    b := temp;
selesai;

prosedur processlogic(limit: integer);
variabel
    counter: integer;
    localbool: boolean;
mulai
    counter := 0;

    selama counter < limit lakukan
        mulai
            counter := counter + 1;

            jika (counter mod 2 = 0) dan (counter > 5) maka
                writeln := 1
            selainitu
                writeln := 1;
        selesai;

    ulangi
        counter := counter - 1;
    sampai counter = 0;

    kasus limit dari
        1: writeln := 1;
        10: writeln := 1;
    akhir;
selesai;

mulai
    gval := 5;
    greal := 10.5 + 2.5;
    isvalid := true;

    myshape.id := 1;
    myshape.active := tidak isvalid;
    myshape.vertices[1] := 100;

```

```

myshape.center.x := 50;

coords[1].y := myshape.center.x * 2;

processlogic(MAX);

gval := factorial(5) + 10;

swap(myshape.id, gval);

untuk i := 1 ke 5 lakukan
mulai
    coords[i].x := i * i;
    writeln := 1;
selesai;

writeln := 1;
selesai.

```

Output

```

===== AST Output =====
Program: TestSemantic
Declarations:
  ConstDecl: MAX
    Value:
      Number: 10
  ConstDecl: PI
    Value:
      Number: 3.14
  ConstDecl: NAME
    Value:
      String: 'Pascals'
  TypeDecl: Vector
    Definition:
      ArrayType: [1..10]
      ElementType:
        BuiltinType: integer
  TypeDecl: Point
    Definition:
      RecordType:
        x:
          BuiltinType: integer
        y:
          BuiltinType: integer
  TypeDecl: Shape
    Definition:
      RecordType:
        id:
          BuiltinType: integer
        vertices:
          CustomType: Vector
        center:
          CustomType: Point
        active:

```

```

        BuiltinType: boolean
VarDecl: gval
  Type:
    BuiltinType: integer
VarDecl: i
  Type:
    BuiltinType: integer
VarDecl: greal
  Type:
    BuiltinType: real
VarDecl: isvalid
  Type:
    BuiltinType: boolean
VarDecl: myshape
  Type:
    CustomType: Shape
VarDecl: coords
  Type:
    ArrayType: [1..5]
    ElementType:
      CustomType: Point
VarDecl: writeln
  Type:
    BuiltinType: integer
Function: factorial
  Parameters:
    n:
      BuiltinType: integer
  Return Type:
    BuiltinType: integer
  Declarations: (none)
  Body:
    Block:
      If:
        Condition:
          BinOp: <=
          Left:
            Variable: n
          Right:
            Number: 1
        Then:
          Assignment:
            Target:
              Variable: factorial
            Value:
              Number: 1
        Else:
          Assignment:
            Target:
              Variable: factorial
            Value:
              BinOp: *
              Left:
                Variable: n
              Right:
                FunctionCall: factorial

```



```

Arguments:
  BinOp: -
  Left:
    Variable: n
  Right:
    Number: 1
Procedure: swap
Parameters:
  a:
    BuiltinType: integer
  b:
    BuiltinType: integer
Declarations:
  VarDecl: temp
    Type:
      BuiltinType: integer
Body:
  Block:
    Assignment:
      Target:
        Variable: temp
      Value:
        Variable: a
    Assignment:
      Target:
        Variable: a
      Value:
        Variable: b
    Assignment:
      Target:
        Variable: b
      Value:
        Variable: temp
Procedure: processlogic
Parameters:
  limit:
    BuiltinType: integer
Declarations:
  VarDecl: counter
    Type:
      BuiltinType: integer
  VarDecl: localbool
    Type:
      BuiltinType: boolean
Body:
  Block:
    Assignment:
      Target:
        Variable: counter
      Value:
        Number: 0
    While:
      Condition:
        BinOp: <
        Left:
          Variable: counter

```

```

    Right:
        Variable: limit
Body:
    Block:
        Assignment:
            Target:
                Variable: counter
            Value:
                BinOp: +
                Left:
                    Variable: counter
                Right:
                    Number: 1
    If:
        Condition:
            BinOp: dan
            Left:
                BinOp: =
                Left:
                    BinOp: mod
                    Left:
                        Variable: counter
                    Right:
                        Number: 2
                Right:
                    Number: 0
            Right:
                BinOp: >
                Left:
                    Variable: counter
                Right:
                    Number: 5
        Then:
            Assignment:
                Target:
                    Variable: writeln
                Value:
                    Number: 1
        Else:
            Assignment:
                Target:
                    Variable: writeln
                Value:
                    Number: 1
Repeat:
    Statements:
        Assignment:
            Target:
                Variable: counter
            Value:
                BinOp: -
                Left:
                    Variable: counter
                Right:
                    Number: 1
Until:

```

```

        BinOp: =
        Left:
            Variable: counter
        Right:
            Number: 0
    Case:
        Selector:
            Variable: limit
        Branches:
            Branch 1:
                Labels:
                    Number: 1
                Statement:
                    Assignment:
                        Target:
                            Variable: writeln
                        Value:
                            Number: 1
            Branch 2:
                Labels:
                    Number: 10
                Statement:
                    Assignment:
                        Target:
                            Variable: writeln
                        Value:
                            Number: 1
    Body:
        Block:
            Assignment:
                Target:
                    Variable: gval
                Value:
                    Number: 5
            Assignment:
                Target:
                    Variable: greal
                Value:
                    BinOp: +
                    Left:
                        Number: 10.5
                    Right:
                        Number: 2.5
            Assignment:
                Target:
                    Variable: isValid
                Value:
                    Boolean: True
            Assignment:
                Target:
                    RecordAccess:
                        Variable:
                            Variable: myshape
                        Field: id
                Value:
                    Number: 1

```

```
Assignment:
  Target:
    RecordAccess:
      Variable:
        Variable: myshape
      Field: active
    Value:
      UnaryOp: tidak
      Operand:
        Variable: isvalid
Assignment:
  Target:
    ArrayAccess:
      Variable:
        RecordAccess:
          Variable:
            Variable: myshape
          Field: vertices
        Index:
          Number: 1
    Value:
      Number: 100
Assignment:
  Target:
    RecordAccess:
      Variable:
        RecordAccess:
          Variable:
            Variable: myshape
          Field: center
        Field: x
    Value:
      Number: 50
Assignment:
  Target:
    RecordAccess:
      Variable:
        ArrayAccess:
          Variable:
            Variable: coords
          Index:
            Number: 1
        Field: y
    Value:
      BinOp: *
      Left:
        RecordAccess:
          Variable:
            RecordAccess:
              Variable:
                Variable: myshape
              Field: center
            Field: x
      Right:
        Number: 2
ProcedureCall: processlogic
```

```

    Arguments:
      Variable: MAX
Assignment:
  Target:
    Variable: gval
  Value:
    BinOp: +
    Left:
      FunctionCall: factorial
      Arguments:
        Number: 5
    Right:
      Number: 10
ProcedureCall: swap
Arguments:
  RecordAccess:
    Variable:
      Variable: myshape
    Field: id
  Variable: gval
For: i
  Start:
    Number: 1
  Direction: ke
  End:
    Number: 5
  Body:
    Block:
      Assignment:
        Target:
          RecordAccess:
            Variable:
              ArrayAccess:
                Variable:
                  Variable: coords
                Index:
                  Variable: i
              Field: x
            Value:
              BinOp: *
              Left:
                Variable: i
              Right:
                Variable: i
          Assignment:
            Target:
              Variable: writeln
            Value:
              Number: 1
Assignment:
  Target:
    Variable: writeln
  Value:
    Number: 1
===== Semantic Analysis Output =====

```

=====							
=====							
TAB (Identifier Table)							
=====							
idx	identifiers	link	obj	type	ref	nrm	lev
adr	init						

0	dan	-1	[6] keyword	[0] undef	-1	0	-1
0	1						
1	larik	0	[6] keyword	[0] undef	-1	0	-1
0	1						
2	mulai	1	[6] keyword	[0] undef	-1	0	-1
0	1						
3	kasus	2	[6] keyword	[0] undef	-1	0	-1
0	1						
4	konstanta	3	[6] keyword	[0] undef	-1	0	-1
0	1						
5	div	4	[6] keyword	[0] undef	-1	0	-1
0	1						
6	turunke	5	[6] keyword	[0] undef	-1	0	-1
0	1						
7	lakukan	6	[6] keyword	[0] undef	-1	0	-1
0	1						
8	selainitu	7	[6] keyword	[0] undef	-1	0	-1
0	1						
9	selesai	8	[6] keyword	[0] undef	-1	0	-1
0	1						
10	untuk	9	[6] keyword	[0] undef	-1	0	-1
0	1						
11	fungsi	10	[6] keyword	[0] undef	-1	0	-1
0	1						
12	jika	11	[6] keyword	[0] undef	-1	0	-1
0	1						
13	mod	12	[6] keyword	[0] undef	-1	0	-1
0	1						
14	tidak	13	[6] keyword	[0] undef	-1	0	-1
0	1						
15	dari	14	[6] keyword	[0] undef	-1	0	-1
0	1						
16	atau	15	[6] keyword	[0] undef	-1	0	-1
0	1						
17	prosedur	16	[6] keyword	[0] undef	-1	0	-1
0	1						
18	program	17	[6] keyword	[0] undef	-1	0	-1
0	1						
19	rekaman	18	[6] keyword	[0] undef	-1	0	-1
0	1						
20	ulangi	19	[6] keyword	[0] undef	-1	0	-1
0	1						
21	string	20	[6] keyword	[0] undef	-1	0	-1
0	1						
22	maka	21	[6] keyword	[0] undef	-1	0	-1
0	1						
23	ke	22	[6] keyword	[0] undef	-1	0	-1

0	1						
24	tipe	23	[6] keyword	[0] undef	-1	0	-1
0	1						
25	sampai	24	[6] keyword	[0] undef	-1	0	-1
0	1						
26	var	25	[6] keyword	[0] undef	-1	0	-1
0	1						
27	selama	26	[6] keyword	[0] undef	-1	0	-1
0	1						
28	packed	27	[6] keyword	[0] undef	-1	0	-1
0	1						
29	TestSemantic	-1	[5] program	[0] undef	-1	1	0
0	1						
30	MAX	29	[0] konstanta	[1] int	-1	1	0
10	1						
31	PI	30	[0] konstanta	[2] real	-1	1	0
3	1						
32	NAME	31	[0] konstanta	[7] string	-1	1	0
7	1						
33	Vector	32	[2] tipe	[5] array	0	1	0
10	1						
34	x	-1	[1] variabel	[1] int	-1	1	1
0	1						
35	y	34	[1] variabel	[1] int	-1	1	1
1	1						
36	Point	33	[2] tipe	[6] record	2	1	0
2	1						
37	id	-1	[1] variabel	[1] int	-1	1	1
0	1						
38	vertices	37	[1] variabel	[5] array	0	1	1
1	1						
39	center	38	[1] variabel	[6] record	2	1	1
11	1						
40	active	39	[1] variabel	[3] boolean	-1	1	1
13	1						
41	Shape	36	[2] tipe	[6] record	3	1	0
14	1						
42	gval	41	[1] variabel	[1] int	-1	1	0
5	1						
43	i	42	[1] variabel	[1] int	-1	1	0
6	1						
44	greal	43	[1] variabel	[2] real	-1	1	0
7	1						
45	isvalid	44	[1] variabel	[3] boolean	-1	1	0
8	1						
46	myshape	45	[1] variabel	[6] record	3	1	0
9	1						
47	coords	46	[1] variabel	[5] array	1	1	0
23	1						
48	writeln	47	[1] variabel	[1] int	-1	1	0
33	1						
49	factorial	48	[4] fungsi	[1] int	4	1	0
0	1						
50	n	-1	[1] variabel	[1] int	-1	1	1
5	1						
51	swap	49	[3] prosedur	[0] undef	5	1	0

```

0      1
52    a      -1    [1] variabel    [1] int      -1    0    1
5      1
53    b      52    [1] variabel    [1] int      -1    0    1
6      1
54    temp   53    [1] variabel    [1] int      -1    1    1
7      1
55    processlogic 51    [3] prosedur    [0] undef    6    1    0
0      1
56    limit  -1    [1] variabel    [1] int      -1    1    1
5      1
57    counter 56    [1] variabel    [1] int      -1    1    1
6      1
58    localbool 57    [1] variabel    [3] boolean  -1    1    1
7      0

```

```

=====
BTAB (Block Table)
=====

```

blocks	last	lpar	psze	vsze
0	28	0	0	0
1	55	-1	0	29
2	35	0	0	2
3	40	0	0	14
4	50	50	1	0
5	54	53	2	1
6	58	56	1	2

```

=====
==
ATAB (Array Table)
=====
==

```

arrays	xtyp	etyp	eref	low	high	elsz	size
0	[1] int	[1] int	-1	1	10	1	10
1	[1] int	[6] record	2	1	5	2	10

```

===== Decorated AST Output =====

```

Program: TestSemantic

Declarations:

ConstDecl: MAX -> index:30, type:integer, lev:0

Value:

Number: 10 -> type:integer

ConstDecl: PI -> index:31, type:real, lev:0

Value:

Number: 3.14 -> type:real

ConstDecl: NAME -> index:32, type:string, lev:0

Value:

String: 'Pascals' -> type:string

TypeDecl: Vector -> index:33, type:larik, lev:0

Definition:


```

    ArrayType: [1..10] -> index:0, type:larik
    ElementType:
        BuiltinType: integer -> type:integer
TypeDecl: Point -> index:36, type:rekaman, lev:0
Definition:
    RecordType: -> index:2, type:rekaman
    x:
        BuiltinType: integer -> type:integer
    y:
        BuiltinType: integer -> type:integer
TypeDecl: Shape -> index:41, type:rekaman, lev:0
Definition:
    RecordType: -> index:3, type:rekaman
    id:
        BuiltinType: integer -> type:integer
    vertices:
        CustomType: Vector -> index:33, type:larik, lev:0
    center:
        CustomType: Point -> index:36, type:rekaman, lev:0
    active:
        BuiltinType: boolean -> type:boolean
VarDecl: gval -> index:42, type:integer, lev:0
Type:
    BuiltinType: integer -> type:integer
VarDecl: i -> index:43, type:integer, lev:0
Type:
    BuiltinType: integer -> type:integer
VarDecl: greal -> index:44, type:real, lev:0
Type:
    BuiltinType: real -> type:real
VarDecl: isvalid -> index:45, type:boolean, lev:0
Type:
    BuiltinType: boolean -> type:boolean
VarDecl: myshape -> index:46, type:rekaman, lev:0
Type:
    CustomType: Shape -> index:41, type:rekaman, lev:0
VarDecl: coords -> index:47, type:larik, lev:0
Type:
    ArrayType: [1..5] -> index:1, type:larik
    ElementType:
        CustomType: Point -> index:36, type:rekaman, lev:0
VarDecl: writeln -> index:48, type:integer, lev:0
Type:
    BuiltinType: integer -> type:integer
Function: factorial -> index:49, type:integer, lev:0
Parameters:
    n:
        BuiltinType: integer -> type:integer
Return Type:
    BuiltinType: integer -> type:integer
Declarations: (none)
Body:
    Block: -> index:4, lev:1
    If: -> lev:1
    Condition:
        BinOp: <= -> type:boolean

```

```

        Left:
            Variable: n -> index:50, type:integer
        Right:
            Number: 1 -> type:integer
    Then:
        Assignment: -> type:void
        Target:
            Variable: factorial -> index:49, type:integer, lev:0
        Value:
            Number: 1 -> type:integer
    Else:
        Assignment: -> type:void
        Target:
            Variable: factorial -> index:49, type:integer, lev:0
        Value:
            BinOp: * -> type:integer
            Left:
                Variable: n -> index:50, type:integer
            Right:
                FunctionCall: factorial -> index:49, type:integer,
lev:0
                Arguments:
                    BinOp: - -> type:integer
                    Left:
                        Variable: n -> index:50, type:integer
                    Right:
                        Number: 1 -> type:integer
Procedure: swap -> index:51, lev:0
    Parameters:
        a:
            BuiltinType: integer -> type:integer
        b:
            BuiltinType: integer -> type:integer
    Declarations:
        VarDecl: temp -> index:54, type:integer, lev:1
        Type:
            BuiltinType: integer -> type:integer
    Body:
        Block: -> index:5, lev:1
        Assignment: -> type:void
        Target:
            Variable: temp -> index:54, type:integer, lev:1
        Value:
            Variable: a -> index:52, type:integer
        Assignment: -> type:void
        Target:
            Variable: a -> index:52, type:integer, lev:1
        Value:
            Variable: b -> index:53, type:integer
        Assignment: -> type:void
        Target:
            Variable: b -> index:53, type:integer, lev:1
        Value:
            Variable: temp -> index:54, type:integer
Procedure: processlogic -> index:55, lev:0
    Parameters:

```

```

    limit:
      BuiltinType: integer -> type:integer
Declarations:
  VarDecl: counter -> index:57, type:integer, lev:1
    Type:
      BuiltinType: integer -> type:integer
  VarDecl: localbool -> index:58, type:boolean, lev:1
    Type:
      BuiltinType: boolean -> type:boolean
Body:
  Block: -> index:6, lev:1
    Assignment: -> type:void
      Target:
        Variable: counter -> index:57, type:integer, lev:1
      Value:
        Number: 0 -> type:integer
  While: -> lev:1
    Condition:
      BinOp: < -> type:boolean
      Left:
        Variable: counter -> index:57, type:integer
      Right:
        Variable: limit -> index:56, type:integer
    Body:
      Block: -> index:6, lev:1
        Assignment: -> type:void
          Target:
            Variable: counter -> index:57, type:integer, lev:1
          Value:
            BinOp: + -> type:integer
            Left:
              Variable: counter -> index:57, type:integer
            Right:
              Number: 1 -> type:integer
      If: -> lev:1
        Condition:
          BinOp: dan -> type:boolean
          Left:
            BinOp: = -> type:boolean
            Left:
              BinOp: mod -> type:integer
              Left:
                Variable: counter -> index:57, type:integer
              Right:
                Number: 2 -> type:integer
            Right:
              Number: 0 -> type:integer
          Right:
            BinOp: > -> type:boolean
            Left:
              Variable: counter -> index:57, type:integer
            Right:
              Number: 5 -> type:integer
        Then:
          Assignment: -> type:void
            Target:

```

```

        Variable: writeln -> index:48, type:integer, lev:0
    Value:
        Number: 1 -> type:integer
    Else:
        Assignment: -> type:void
        Target:
            Variable: writeln -> index:48, type:integer, lev:0
        Value:
            Number: 1 -> type:integer
Repeat: -> lev:1
    Statements:
        Assignment: -> type:void
        Target:
            Variable: counter -> index:57, type:integer, lev:1
        Value:
            BinOp: - -> type:integer
            Left:
                Variable: counter -> index:57, type:integer
            Right:
                Number: 1 -> type:integer
    Until:
        BinOp: = -> type:boolean
        Left:
            Variable: counter -> index:57, type:integer
        Right:
            Number: 0 -> type:integer
Case: -> lev:1
    Selector:
        Variable: limit -> index:56, type:integer
    Branches:
        Branch 1:
            Labels:
                Number: 1 -> type:integer
            Statement:
                Assignment: -> type:void
                Target:
                    Variable: writeln -> index:48, type:integer, lev:0
                Value:
                    Number: 1 -> type:integer
        Branch 2:
            Labels:
                Number: 10 -> type:integer
            Statement:
                Assignment: -> type:void
                Target:
                    Variable: writeln -> index:48, type:integer, lev:0
                Value:
                    Number: 1 -> type:integer
Body:
    Block: -> index:1, lev:0
    Assignment: -> type:void
    Target:
        Variable: gval -> index:42, type:integer, lev:0
    Value:
        Number: 5 -> type:integer
    Assignment: -> type:void

```

```

Target:
  Variable: greal -> index:44, type:real, lev:0
Value:
  BinOp: + -> type:real
    Left:
      Number: 10.5 -> type:real
    Right:
      Number: 2.5 -> type:real
Assignment: -> type:void
Target:
  Variable: isvalid -> index:45, type:boolean, lev:0
Value:
  Boolean: True -> type:boolean
Assignment: -> type:void
Target:
  RecordAccess: -> index:3, lev:37
  Variable:
    Variable: myshape -> index:46, type:rekaman
    Field: id
Value:
  Number: 1 -> type:integer
Assignment: -> type:void
Target:
  RecordAccess: -> index:3, lev:40
  Variable:
    Variable: myshape -> index:46, type:rekaman
    Field: active
Value:
  UnaryOp: tidak -> type:boolean
  Operand:
    Variable: isvalid -> index:45, type:boolean
Assignment: -> type:void
Target:
  ArrayAccess: -> index:0
  Variable:
    RecordAccess: -> index:3, lev:38
    Variable:
      Variable: myshape -> index:46, type:rekaman
      Field: vertices
    Index:
      Number: 1 -> type:integer
Value:
  Number: 100 -> type:integer
Assignment: -> type:void
Target:
  RecordAccess: -> index:2, lev:34
  Variable:
    RecordAccess: -> index:3, lev:39
    Variable:
      Variable: myshape -> index:46, type:rekaman
      Field: center
    Field: x
Value:
  Number: 50 -> type:integer
Assignment: -> type:void
Target:

```

```

RecordAccess: -> index:2, lev:35
Variable:
  ArrayAccess: -> index:1
  Variable:
    Variable: coords -> index:47, type:larik
  Index:
    Number: 1 -> type:integer
  Field: y
Value:
  BinOp: * -> type:integer
  Left:
    RecordAccess: -> index:2, lev:34
    Variable:
      RecordAccess: -> index:3, lev:39
      Variable:
        Variable: myshape -> index:46, type:rekaman
      Field: center
    Field: x
  Right:
    Number: 2 -> type:integer
ProcedureCall: processlogic -> index:55, lev:0
Arguments:
  Variable: MAX -> index:30, type:integer
Assignment: -> type:void
Target:
  Variable: gval -> index:42, type:integer, lev:0
Value:
  BinOp: + -> type:integer
  Left:
    FunctionCall: factorial -> index:49, type:integer, lev:0
    Arguments:
      Number: 5 -> type:integer
  Right:
    Number: 10 -> type:integer
ProcedureCall: swap -> index:51, lev:0
Arguments:
  RecordAccess: -> index:3, lev:37
  Variable:
    Variable: myshape -> index:46, type:rekaman
  Field: id
  Variable: gval -> index:42, type:integer, lev:0
For: i -> lev:0
Start:
  Number: 1 -> type:integer
Direction: ke
End:
  Number: 5 -> type:integer
Body:
  Block: -> index:1, lev:0
  Assignment: -> type:void
  Target:
    RecordAccess: -> index:2, lev:34
    Variable:
      ArrayAccess: -> index:1
      Variable:
        Variable: coords -> index:47, type:larik

```

```

        Index:
          Variable: i  -> index:43, type:integer
        Field: x
      Value:
        BinOp: *  -> type:integer
        Left:
          Variable: i  -> index:43, type:integer
        Right:
          Variable: i  -> index:43, type:integer
      Assignment: -> type:void
      Target:
        Variable: writeln -> index:48, type:integer, lev:0
      Value:
        Number: 1  -> type:integer
    Assignment: -> type:void
    Target:
      Variable: writeln -> index:48, type:integer, lev:0
    Value:
      Number: 1  -> type:integer

```

Kasus 8

Input file: test8.pas

```

program ComprehensiveTest;

konstanta
  MAXSIZE = 100;
  PIVALUE = 3.14159;
  APPNAME = 'SemanticTester';
  NEWLINE = '\n';

tipe
  IndexRange = integer;

  Matrix = larik [1..10] dari larik [1..10] dari integer;

  Student = rekaman
    id: integer;
    name: larik [1..50] dari char;
    gpa: real;
    active: boolean;
  selesai;

  Classroom = rekaman
    roomnumber: integer;
    students: larik [1..30] dari Student;
    capacity: integer;
  selesai;

  IntArray = larik [0..99] dari integer;

variabel
  i, j, k: integer;

```

```

    x, y, z: real;
    flag, done: boolean;
    grid: Matrix;
    class: Classroom;
    scores: IntArray;
    tempstudent: Student;

fungsi calculate(a: integer; b: real; var result: real): boolean;
variabel
    temp: real;
mulai
    temp := a * b;
    result := temp + PIValue;

    jika result > 100.0 maka
        calculate := true
    selainitu
        calculate := false;
selesai;

fungsi sumarray(arr: IntArray; size: integer): integer;
variabel
    total, idx: integer;
mulai
    total := 0;
    untuk idx := 0 ke size - 1 lakukan
        total := total + arr[idx];
    sumarray := total;
selesai;

prosedur swapstudents(var s1, s2: Student);
variabel
    temp: Student;
mulai
    temp := s1;
    s1 := s2;
    s2 := temp;
selesai;

prosedur updateclassroom(var c: Classroom; room: integer; cap: integer);
mulai
    c.roomnumber := room;
    c.capacity := cap;
selesai;

prosedur testcomplexaccess;
variabel
    idx: integer;
mulai
    class.students[1].id := 1001;
    class.students[1].gpa := 3.75;
    class.students[1].active := true;

    tempstudent := class.students[1];

    class.students[1].name[1] := 'J';

```



```

class.students[1].name[2] := 'o';
class.students[1].name[3] := 'h';
class.students[1].name[4] := 'n';
selesai;

prosedur testcontrolflow(n: integer);
variabel
  counter, value: integer;
  condition: boolean;
mulai
  jika n > 0 maka
    mulai
      value := n * 2;
    selesai
  selainitu
    mulai
      value := 0;
    selesai;

  counter := 0;
  selama counter < n lakukan
    mulai
      counter := counter + 1;
    selesai;

  untuk i := 1 ke 10 lakukan
    mulai
      scores[i] := i * i;
    selesai;

  untuk i := 10 turunke 1 lakukan
    mulai
      value := scores[i];
    selesai;

  counter := 0;
  ulangi
    counter := counter + 1;
  sampai counter >= n;

  kasus n dari
    1: value := 10;
    2: value := 20;
    5: value := 50;
    10: value := 100;
  akhir;
selesai;

fungsi complexexpression(a, b, c: integer): real;
variabel
  result: real;
mulai
  result := (a + b) * c / 2.0;

  result := result + ((a - b) * (c + 5)) / 3.0;

```

```

    complexexpression := result;
selesai;

fungsi testboolean(x, y: integer; flag: boolean): boolean;
variabel
    temp: boolean;
mulai
    temp := (x > y) dan (x < 100);

    temp := ((x >= 0) dan (y <= 10)) atau flag;

    temp := tidak temp;

    testboolean := temp atau ((x = y) dan (x <> 0));
selesai;

prosedur testnestedcalls(dummy: integer);
variabel
    success: boolean;
    total: integer;
    res: real;
mulai
    total := sumarray(scores, 50);

    success := calculate(10, 2.5, res);

    jika calculate(5, 3.0, res) maka
        total := sumarray(scores, 10);
selesai;

prosedur testarrays(dummy: integer);
variabel
    a, b: integer;
mulai
    untuk a := 0 ke 99 lakukan
        scores[a] := a;

    untuk a := 1 ke 10 lakukan
        untuk b := 1 ke 10 lakukan
            grid[a][b] := a * b;
selesai;

prosedur testrecords(dummy: integer);
variabel
    s1, s2: Student;
mulai
    s1.id := 100;
    s1.gpa := 3.5;
    s1.active := true;

    s2 := s1;

    s2.id := 200;

    swapstudents(s1, s2);
selesai;

```

```

prosedur testedgecases(dummy: integer);
variabel
    minval, maxval: integer;
    epsilon: real;
mulai
    minval := 0;
    maxval := MAXSIZE;

    epsilon := 0.0001;

    minval := -100;

    scores[0] := minval;
    scores[99] := maxval;

    grid[1][1] := 1;
    grid[10][10] := 100;
selesai;

mulai
    i := 0;
    x := 0.0;
    flag := false;

    i := MAXSIZE;
    x := PIVALUE;

    testcomplexaccess;
    testcontrolflow(10);
    testnestedcalls(i);
    testarrays(i);
    testrecords(i);
    testedgecases(i);

    x := complexexpression(5, 3, 2);
    flag := testboolean(10, 20, true);

    i := sumarray(scores, 100);

    jika calculate(i, x, y) maka
        mulai
            updateclassroom(class, 101, 30);
        selesai;
    selesai.

```

Output

```

===== AST Output =====
Program: ComprehensiveTest
Declarations:
  ConstDecl: MAXSIZE
    Value:
      Number: 100
  ConstDecl: PIVALUE

```

```
Value:
  Number: 3.14159
ConstDecl: APPNAME
Value:
  String: 'SemanticTester'
ConstDecl: NEWLINE
Value:
  String: '\n'
TypeDecl: IndexRange
Definition:
  BuiltinType: integer
TypeDecl: Matrix
Definition:
  ArrayType: [1..10]
  ElementType:
    ArrayType: [1..10]
    ElementType:
      BuiltinType: integer
TypeDecl: Student
Definition:
  RecordType:
    id:
      BuiltinType: integer
    name:
      ArrayType: [1..50]
      ElementType:
        BuiltinType: char
    gpa:
      BuiltinType: real
    active:
      BuiltinType: boolean
TypeDecl: Classroom
Definition:
  RecordType:
    roomnumber:
      BuiltinType: integer
    students:
      ArrayType: [1..30]
      ElementType:
        CustomType: Student
    capacity:
      BuiltinType: integer
TypeDecl: IntArray
Definition:
  ArrayType: [0..99]
  ElementType:
    BuiltinType: integer
VarDecl: i
Type:
  BuiltinType: integer
VarDecl: j
Type:
  BuiltinType: integer
VarDecl: k
Type:
  BuiltinType: integer
```

```
VarDecl: x
  Type:
    BuiltinType: real
VarDecl: y
  Type:
    BuiltinType: real
VarDecl: z
  Type:
    BuiltinType: real
VarDecl: flag
  Type:
    BuiltinType: boolean
VarDecl: done
  Type:
    BuiltinType: boolean
VarDecl: grid
  Type:
    CustomType: Matrix
VarDecl: class
  Type:
    CustomType: Classroom
VarDecl: scores
  Type:
    CustomType: IntArray
VarDecl: tempstudent
  Type:
    CustomType: Student
Function: calculate
  Parameters:
    a:
      BuiltinType: integer
    b:
      BuiltinType: real
    result:
      BuiltinType: real
  Return Type:
    BuiltinType: boolean
  Declarations:
    VarDecl: temp
      Type:
        BuiltinType: real
  Body:
    Block:
      Assignment:
        Target:
          Variable: temp
        Value:
          BinOp: *
            Left:
              Variable: a
            Right:
              Variable: b
      Assignment:
        Target:
          Variable: result
        Value:
```

```

        BinOp: +
        Left:
            Variable: temp
        Right:
            Variable: PIVALUE
    If:
        Condition:
            BinOp: >
            Left:
                Variable: result
            Right:
                Number: 100.0
        Then:
            Assignment:
                Target:
                    Variable: calculate
                Value:
                    Boolean: True
        Else:
            Assignment:
                Target:
                    Variable: calculate
                Value:
                    Boolean: False
Function: sumarray
Parameters:
    arr:
        CustomType: IntArray
    size:
        BuiltinType: integer
Return Type:
    BuiltinType: integer
Declarations:
    VarDecl: total
        Type:
            BuiltinType: integer
    VarDecl: idx
        Type:
            BuiltinType: integer
Body:
    Block:
        Assignment:
            Target:
                Variable: total
            Value:
                Number: 0
    For: idx
        Start:
            Number: 0
        Direction: ke
    End:
        BinOp: -
        Left:
            Variable: size
        Right:
            Number: 1

```

```

        Body:
        Assignment:
        Target:
        Variable: total
        Value:
        BinOp: +
        Left:
        Variable: total
        Right:
        ArrayAccess:
        Variable:
        Variable: arr
        Index:
        Variable: idx
    Assignment:
    Target:
    Variable: sumarray
    Value:
    Variable: total
Procedure: swapstudents
Parameters:
    s1:
    CustomType: Student
    s2:
    CustomType: Student
Declarations:
    VarDecl: temp
    Type:
    CustomType: Student
Body:
    Block:
    Assignment:
    Target:
    Variable: temp
    Value:
    Variable: s1
    Assignment:
    Target:
    Variable: s1
    Value:
    Variable: s2
    Assignment:
    Target:
    Variable: s2
    Value:
    Variable: temp
Procedure: updateclassroom
Parameters:
    c:
    CustomType: Classroom
    room:
    BuiltinType: integer
    cap:
    BuiltinType: integer
Declarations: (none)
Body:

```

```

Block:
  Assignment:
    Target:
      RecordAccess:
        Variable:
          Variable: c
          Field: roomnumber
      Value:
        Variable: room
  Assignment:
    Target:
      RecordAccess:
        Variable:
          Variable: c
          Field: capacity
      Value:
        Variable: cap
Procedure: testcomplexaccess
Parameters: (none)
Declarations:
  VarDecl: idx
  Type:
    BuiltinType: integer
Body:
  Block:
    Assignment:
      Target:
        RecordAccess:
          Variable:
            ArrayAccess:
              Variable:
                RecordAccess:
                  Variable:
                    Variable: class
                    Field: students
            Index:
              Number: 1
          Field: id
      Value:
        Number: 1001
    Assignment:
      Target:
        RecordAccess:
          Variable:
            ArrayAccess:
              Variable:
                RecordAccess:
                  Variable:
                    Variable: class
                    Field: students
            Index:
              Number: 1
          Field: gpa
      Value:
        Number: 3.75
    Assignment:

```



```
Target:
  RecordAccess:
    Variable:
      ArrayAccess:
        Variable:
          RecordAccess:
            Variable:
              Variable: class
            Field: students
          Index:
            Number: 1
        Field: active
    Value:
      Boolean: True
Assignment:
  Target:
    Variable: tempstudent
  Value:
    ArrayAccess:
      Variable:
        RecordAccess:
          Variable:
            Variable: class
          Field: students
        Index:
          Number: 1
Assignment:
  Target:
    ArrayAccess:
      Variable:
        RecordAccess:
          Variable:
            ArrayAccess:
              Variable:
                RecordAccess:
                  Variable:
                    Variable: class
                  Field: students
                Index:
                  Number: 1
              Field: name
            Index:
              Number: 1
          Value:
            Char: 'J'
Assignment:
  Target:
    ArrayAccess:
      Variable:
        RecordAccess:
          Variable:
            ArrayAccess:
              Variable:
                RecordAccess:
                  Variable:
                    Variable: class
```

```

        Field: students
        Index:
        Number: 1
    Field: name
    Index:
    Number: 2
Value:
    Char: 'o'
Assignment:
    Target:
        ArrayAccess:
            Variable:
                RecordAccess:
                    Variable:
                        ArrayAccess:
                            Variable:
                                RecordAccess:
                                    Variable:
                                        Variable: class
                                        Field: students
                                    Index:
                                    Number: 1
                                Field: name
                                Index:
                                Number: 3
                            Value:
                                Char: 'h'
                        Assignment:
                            Target:
                                ArrayAccess:
                                    Variable:
                                        RecordAccess:
                                            Variable:
                                                ArrayAccess:
                                                    Variable:
                                                        RecordAccess:
                                                            Variable:
                                                                Variable: class
                                                                Field: students
                                                            Index:
                                                            Number: 1
                                                        Field: name
                                                        Index:
                                                        Number: 4
                                                    Value:
                                                        Char: 'n'
                    Procedure: testcontrolflow
                    Parameters:
                        n:
                            BuiltinType: integer
                    Declarations:
                        VarDecl: counter
                        Type:
                            BuiltinType: integer
                        VarDecl: value
                        Type:

```

```

    BuiltinType: integer
VarDecl: condition
    Type:
        BuiltinType: boolean
Body:
    Block:
        If:
            Condition:
                BinOp: >
                Left:
                    Variable: n
                Right:
                    Number: 0
            Then:
                Block:
                    Assignment:
                        Target:
                            Variable: value
                        Value:
                            BinOp: *
                            Left:
                                Variable: n
                            Right:
                                Number: 2
            Else:
                Block:
                    Assignment:
                        Target:
                            Variable: value
                        Value:
                            Number: 0
        Assignment:
            Target:
                Variable: counter
            Value:
                Number: 0
        While:
            Condition:
                BinOp: <
                Left:
                    Variable: counter
                Right:
                    Variable: n
            Body:
                Block:
                    Assignment:
                        Target:
                            Variable: counter
                        Value:
                            BinOp: +
                            Left:
                                Variable: counter
                            Right:
                                Number: 1
        For: i
            Start:

```

```

    Number: 1
    Direction: ke
    End:
    Number: 10
    Body:
    Block:
    Assignment:
    Target:
    ArrayAccess:
    Variable:
    Variable: scores
    Index:
    Variable: i
    Value:
    BinOp: *
    Left:
    Variable: i
    Right:
    Variable: i
For: i
    Start:
    Number: 10
    Direction: turunke
    End:
    Number: 1
    Body:
    Block:
    Assignment:
    Target:
    Variable: value
    Value:
    ArrayAccess:
    Variable:
    Variable: scores
    Index:
    Variable: i
Assignment:
    Target:
    Variable: counter
    Value:
    Number: 0
Repeat:
    Statements:
    Assignment:
    Target:
    Variable: counter
    Value:
    BinOp: +
    Left:
    Variable: counter
    Right:
    Number: 1
Until:
    BinOp: >=
    Left:
    Variable: counter

```

```

        Right:
            Variable: n
    Case:
        Selector:
            Variable: n
        Branches:
            Branch 1:
                Labels:
                    Number: 1
                Statement:
                    Assignment:
                        Target:
                            Variable: value
                        Value:
                            Number: 10
            Branch 2:
                Labels:
                    Number: 2
                Statement:
                    Assignment:
                        Target:
                            Variable: value
                        Value:
                            Number: 20
            Branch 3:
                Labels:
                    Number: 5
                Statement:
                    Assignment:
                        Target:
                            Variable: value
                        Value:
                            Number: 50
            Branch 4:
                Labels:
                    Number: 10
                Statement:
                    Assignment:
                        Target:
                            Variable: value
                        Value:
                            Number: 100
    Function: complexexpression
    Parameters:
        a:
            BuiltinType: integer
        b:
            BuiltinType: integer
        c:
            BuiltinType: integer
    Return Type:
        BuiltinType: real
    Declarations:
        VarDecl: result
        Type:
            BuiltinType: real

```

```

Body:
  Block:
    Assignment:
      Target:
        Variable: result
      Value:
        BinOp: /
        Left:
          BinOp: *
          Left:
            BinOp: +
            Left:
              Variable: a
            Right:
              Variable: b
          Right:
            Variable: c
        Right:
          Number: 2.0
    Assignment:
      Target:
        Variable: result
      Value:
        BinOp: +
        Left:
          Variable: result
        Right:
          BinOp: /
          Left:
            BinOp: *
            Left:
              BinOp: -
              Left:
                Variable: a
              Right:
                Variable: b
            Right:
              BinOp: +
              Left:
                Variable: c
              Right:
                Number: 5
          Right:
            Number: 3.0
    Assignment:
      Target:
        Variable: complexexpression
      Value:
        Variable: result
Function: testboolean
Parameters:
  x:
    BuiltinType: integer
  y:
    BuiltinType: integer
  flag:

```

```

    BuiltinType: boolean
Return Type:
    BuiltinType: boolean
Declarations:
    VarDecl: temp
        Type:
            BuiltinType: boolean
Body:
    Block:
        Assignment:
            Target:
                Variable: temp
            Value:
                BinOp: dan
                Left:
                    BinOp: >
                    Left:
                        Variable: x
                    Right:
                        Variable: y
                Right:
                    BinOp: <
                    Left:
                        Variable: x
                    Right:
                        Number: 100
        Assignment:
            Target:
                Variable: temp
            Value:
                BinOp: atau
                Left:
                    BinOp: dan
                    Left:
                        BinOp: >=
                        Left:
                            Variable: x
                        Right:
                            Number: 0
                    Right:
                        BinOp: <=
                        Left:
                            Variable: y
                        Right:
                            Number: 10
                Right:
                    Variable: flag
        Assignment:
            Target:
                Variable: temp
            Value:
                UnaryOp: tidak
                Operand:
                    Variable: temp
        Assignment:
            Target:

```

```

        Variable: testboolean
    Value:
        BinOp: atau
        Left:
            Variable: temp
        Right:
            BinOp: dan
            Left:
                BinOp: =
                Left:
                    Variable: x
                Right:
                    Variable: y
            Right:
                BinOp: <>
                Left:
                    Variable: x
                Right:
                    Number: 0
Procedure: testnestedcalls
Parameters:
    dummy:
        BuiltinType: integer
Declarations:
    VarDecl: success
        Type:
            BuiltinType: boolean
    VarDecl: total
        Type:
            BuiltinType: integer
    VarDecl: res
        Type:
            BuiltinType: real
Body:
    Block:
        Assignment:
            Target:
                Variable: total
            Value:
                FunctionCall: sumarray
                Arguments:
                    Variable: scores
                    Number: 50
        Assignment:
            Target:
                Variable: success
            Value:
                FunctionCall: calculate
                Arguments:
                    Number: 10
                    Number: 2.5
                    Variable: res
    If:
        Condition:
            FunctionCall: calculate
            Arguments:

```



```

        Number: 5
        Number: 3.0
        Variable: res
    Then:
        Assignment:
            Target:
                Variable: total
            Value:
                FunctionCall: sumarray
                Arguments:
                    Variable: scores
                    Number: 10
Procedure: testarrays
Parameters:
    dummy:
        BuiltinType: integer
Declarations:
    VarDecl: a
        Type:
            BuiltinType: integer
    VarDecl: b
        Type:
            BuiltinType: integer
Body:
    Block:
        For: a
            Start:
                Number: 0
            Direction: ke
            End:
                Number: 99
            Body:
                Assignment:
                    Target:
                        ArrayAccess:
                            Variable: scores
                            Index:
                                Variable: a
                        Value:
                            Variable: a
        For: a
            Start:
                Number: 1
            Direction: ke
            End:
                Number: 10
            Body:
                For: b
                    Start:
                        Number: 1
                    Direction: ke
                    End:
                        Number: 10
                    Body:
                        Assignment:

```

```

        Target:
        ArrayAccess:
        Variable:
        ArrayAccess:
        Variable:
        Variable: grid
        Index:
        Variable: a
        Index:
        Variable: b
    Value:
    BinOp: *
    Left:
    Variable: a
    Right:
    Variable: b
Procedure: testrecords
Parameters:
    dummy:
        BuiltinType: integer
Declarations:
    VarDecl: s1
    Type:
        CustomType: Student
    VarDecl: s2
    Type:
        CustomType: Student
Body:
    Block:
        Assignment:
        Target:
        RecordAccess:
        Variable:
        Variable: s1
        Field: id
        Value:
        Number: 100
        Assignment:
        Target:
        RecordAccess:
        Variable:
        Variable: s1
        Field: gpa
        Value:
        Number: 3.5
        Assignment:
        Target:
        RecordAccess:
        Variable:
        Variable: s1
        Field: active
        Value:
        Boolean: True
        Assignment:
        Target:
        Variable: s2

```

```

    Value:
      Variable: s1
  Assignment:
    Target:
      RecordAccess:
        Variable:
          Variable: s2
          Field: id
      Value:
        Number: 200
  ProcedureCall: swapstudents
  Arguments:
    Variable: s1
    Variable: s2
Procedure: testedgecases
Parameters:
  dummy:
    BuiltinType: integer
Declarations:
  VarDecl: minval
    Type:
      BuiltinType: integer
  VarDecl: maxval
    Type:
      BuiltinType: integer
  VarDecl: epsilon
    Type:
      BuiltinType: real
Body:
  Block:
    Assignment:
      Target:
        Variable: minval
      Value:
        Number: 0
    Assignment:
      Target:
        Variable: maxval
      Value:
        Variable: MAXSIZE
    Assignment:
      Target:
        Variable: epsilon
      Value:
        Number: 0.0001
    Assignment:
      Target:
        Variable: minval
      Value:
        UnaryOp: -
        Operand:
          Number: 100
    Assignment:
      Target:
        ArrayAccess:
          Variable:

```

```
        Variable: scores
        Index:
        Number: 0
    Value:
        Variable: minval
Assignment:
    Target:
        ArrayAccess:
        Variable:
        Variable: scores
        Index:
        Number: 99
    Value:
        Variable: maxval
Assignment:
    Target:
        ArrayAccess:
        Variable:
        ArrayAccess:
        Variable:
        Variable: grid
        Index:
        Number: 1
    Index:
    Number: 1
    Value:
    Number: 1
Assignment:
    Target:
        ArrayAccess:
        Variable:
        ArrayAccess:
        Variable:
        Variable: grid
        Index:
        Number: 10
    Index:
    Number: 10
    Value:
    Number: 100
Body:
    Block:
        Assignment:
        Target:
        Variable: i
    Value:
    Number: 0
Assignment:
    Target:
    Variable: x
    Value:
    Number: 0.0
Assignment:
    Target:
    Variable: flag
    Value:
```

```
        Boolean: False
Assignment:
  Target:
    Variable: i
  Value:
    Variable: MAXSIZE
Assignment:
  Target:
    Variable: x
  Value:
    Variable: PIVALUE
ProcedureCall: testcomplexaccess
  (no arguments)
ProcedureCall: testcontrolflow
  Arguments:
    Number: 10
ProcedureCall: testnestedcalls
  Arguments:
    Variable: i
ProcedureCall: testarrays
  Arguments:
    Variable: i
ProcedureCall: testrecords
  Arguments:
    Variable: i
ProcedureCall: testedgecases
  Arguments:
    Variable: i
Assignment:
  Target:
    Variable: x
  Value:
    FunctionCall: complexexpression
      Arguments:
        Number: 5
        Number: 3
        Number: 2
Assignment:
  Target:
    Variable: flag
  Value:
    FunctionCall: testboolean
      Arguments:
        Number: 10
        Number: 20
        Boolean: True
Assignment:
  Target:
    Variable: i
  Value:
    FunctionCall: sumarray
      Arguments:
        Variable: scores
        Number: 100
If:
  Condition:
```

```

FunctionCall: calculate
Arguments:
  Variable: i
  Variable: x
  Variable: y
Then:
  Block:
    ProcedureCall: updateclassroom
    Arguments:
      Variable: class
      Number: 101
      Number: 30

```

===== Semantic Analysis Output =====

=====

TAB (Identifier Table)

=====

idx	identifiers		link	obj	type		ref	nrm
lev	adr	init						
0	dan		-1	[6] keyword	[0] undef	-1	0	
-1	0	1						
1	larik		0	[6] keyword	[0] undef	-1	0	
-1	0	1						
2	mulai		1	[6] keyword	[0] undef	-1	0	
-1	0	1						
3	kasus		2	[6] keyword	[0] undef	-1	0	
-1	0	1						
4	konstanta		3	[6] keyword	[0] undef	-1	0	
-1	0	1						
5	div		4	[6] keyword	[0] undef	-1	0	
-1	0	1						
6	turunke		5	[6] keyword	[0] undef	-1	0	
-1	0	1						
7	lakukan		6	[6] keyword	[0] undef	-1	0	
-1	0	1						
8	selainitu		7	[6] keyword	[0] undef	-1	0	
-1	0	1						
9	selesai		8	[6] keyword	[0] undef	-1	0	
-1	0	1						
10	untuk		9	[6] keyword	[0] undef	-1	0	
-1	0	1						
11	fungsi		10	[6] keyword	[0] undef	-1	0	
-1	0	1						
12	jika		11	[6] keyword	[0] undef	-1	0	
-1	0	1						
13	mod		12	[6] keyword	[0] undef	-1	0	
-1	0	1						
14	tidak		13	[6] keyword	[0] undef	-1	0	
-1	0	1						
15	dari		14	[6] keyword	[0] undef	-1	0	
-1	0	1						
16	atau		15	[6] keyword	[0] undef	-1	0	

-1	0	1						
17	prosedur		16	[6] keyword	[0] undef	-1	0	
-1	0	1						
18	program		17	[6] keyword	[0] undef	-1	0	
-1	0	1						
19	rekaman		18	[6] keyword	[0] undef	-1	0	
-1	0	1						
20	ulangi		19	[6] keyword	[0] undef	-1	0	
-1	0	1						
21	string		20	[6] keyword	[0] undef	-1	0	
-1	0	1						
22	maka		21	[6] keyword	[0] undef	-1	0	
-1	0	1						
23	ke		22	[6] keyword	[0] undef	-1	0	
-1	0	1						
24	tipe		23	[6] keyword	[0] undef	-1	0	
-1	0	1						
25	sampai		24	[6] keyword	[0] undef	-1	0	
-1	0	1						
26	var		25	[6] keyword	[0] undef	-1	0	
-1	0	1						
27	selama		26	[6] keyword	[0] undef	-1	0	
-1	0	1						
28	packed		27	[6] keyword	[0] undef	-1	0	
-1	0	1						
29	ComprehensiveTest		-1	[5] program	[0] undef	-1	1	0
0	1							
30	MAXSIZE		29	[0] konstanta	[1] int	-1	1	0
100	1							
31	PIVALUE		30	[0] konstanta	[2] real	-1	1	0
3	1							
32	APPNAME		31	[0] konstanta	[7] string	-1	1	0
14	1							
33	NEWLINE		32	[0] konstanta	[7] string	-1	1	0
2	1							
34	IndexRange		33	[2] tipe	[1] int	-1	1	0
1	1							
35	Matrix		34	[2] tipe	[5] array	1	1	0
100	1							
36	id		-1	[1] variabel	[1] int	-1	1	1
0	1							
37	name		36	[1] variabel	[5] array	2	1	1
1	1							
38	gpa		37	[1] variabel	[2] real	-1	1	1
51	1							
39	active		38	[1] variabel	[3] boolean	-1	1	1
52	1							
40	Student		35	[2] tipe	[6] record	2	1	0
53	1							
41	roomnumber		-1	[1] variabel	[1] int	-1	1	1
0	1							
42	students		41	[1] variabel	[5] array	3	1	1
1	1							
43	capacity		42	[1] variabel	[1] int	-1	1	1
1591	1							
44	Classroom		40	[2] tipe	[6] record	3	1	0

1592	1						
45	IntArray	44	[2] tipe	[5] array	4	1	0
100	1						
46	i	45	[1] variabel	[1] int	-1	1	0
5	1						
47	j	46	[1] variabel	[1] int	-1	1	0
6	0						
48	k	47	[1] variabel	[1] int	-1	1	0
7	0						
49	x	48	[1] variabel	[2] real	-1	1	0
8	1						
50	y	49	[1] variabel	[2] real	-1	1	0
9	1						
51	z	50	[1] variabel	[2] real	-1	1	0
10	0						
52	flag	51	[1] variabel	[3] boolean	-1	1	0
11	1						
53	done	52	[1] variabel	[3] boolean	-1	1	0
12	0						
54	grid	53	[1] variabel	[5] array	1	1	0
13	1						
55	class	54	[1] variabel	[6] record	3	1	0
113	1						
56	scores	55	[1] variabel	[5] array	4	1	0
1705	1						
57	tempstudent	56	[1] variabel	[6] record	2	1	0
1805	1						
58	calculate	57	[4] fungsi	[3] boolean	4	1	0
0	1						
59	a	-1	[1] variabel	[1] int	-1	1	1
5	1						
60	b	59	[1] variabel	[2] real	-1	1	1
6	1						
61	result	60	[1] variabel	[2] real	-1	0	1
7	1						
62	temp	61	[1] variabel	[2] real	-1	1	1
8	1						
63	sumarray	58	[4] fungsi	[1] int	5	1	0
0	1						
64	arr	-1	[1] variabel	[5] array	4	1	1
5	1						
65	size	64	[1] variabel	[1] int	-1	1	1
105	1						
66	total	65	[1] variabel	[1] int	-1	1	1
106	1						
67	idx	66	[1] variabel	[1] int	-1	1	1
107	1						
68	swapstudents	63	[3] prosedur	[0] undef	6	1	0
0	1						
69	s1	-1	[1] variabel	[6] record	2	0	1
5	1						
70	s2	69	[1] variabel	[6] record	2	0	1
6	1						
71	temp	70	[1] variabel	[6] record	2	1	1
7	1						
72	updateclassroom	68	[3] prosedur	[0] undef	7	1	0

0	1							
73	c	-1	[1] variabel	[6] record	3	0	1	
5	1							
74	room	73	[1] variabel	[1] int	-1	1	1	
6	1							
75	cap	74	[1] variabel	[1] int	-1	1	1	
7	1							
76	testcomplexaccess	72	[3] prosedur	[0] undef	8	1	0	
0	1							
77	idx	-1	[1] variabel	[1] int	-1	1	1	
5	0							
78	testcontrolflow	76	[3] prosedur	[0] undef	9	1	0	
0	1							
79	n	-1	[1] variabel	[1] int	-1	1	1	
5	1							
80	counter	79	[1] variabel	[1] int	-1	1	1	
6	1							
81	value	80	[1] variabel	[1] int	-1	1	1	
7	1							
82	condition	81	[1] variabel	[3] boolean	-1	1	1	
8	0							
83	complexexpression	78	[4] fungsi	[2] real	10	1	0	
0	1							
84	a	-1	[1] variabel	[1] int	-1	1	1	
5	1							
85	b	84	[1] variabel	[1] int	-1	1	1	
6	1							
86	c	85	[1] variabel	[1] int	-1	1	1	
7	1							
87	result	86	[1] variabel	[2] real	-1	1	1	
8	1							
88	testboolean	83	[4] fungsi	[3] boolean	11	1	0	
0	1							
89	x	-1	[1] variabel	[1] int	-1	1	1	
5	1							
90	y	89	[1] variabel	[1] int	-1	1	1	
6	1							
91	flag	90	[1] variabel	[3] boolean	-1	1	1	
7	1							
92	temp	91	[1] variabel	[3] boolean	-1	1	1	
8	1							
93	testnestedcalls	88	[3] prosedur	[0] undef	12	1	0	
0	1							
94	dummy	-1	[1] variabel	[1] int	-1	1	1	
5	1							
95	success	94	[1] variabel	[3] boolean	-1	1	1	
6	1							
96	total	95	[1] variabel	[1] int	-1	1	1	
7	1							
97	res	96	[1] variabel	[2] real	-1	1	1	
8	1							
98	testarrays	93	[3] prosedur	[0] undef	13	1	0	
0	1							
99	dummy	-1	[1] variabel	[1] int	-1	1	1	
5	1							
100	a	99	[1] variabel	[1] int	-1	1	1	

6	1							
101	b		100	[1] variabel	[1] int	-1	1	1
7	1							
102	testrecords		98	[3] prosedur	[0] undef	14	1	0
0	1							
103	dummy		-1	[1] variabel	[1] int	-1	1	1
5	1							
104	s1		103	[1] variabel	[6] record	2	1	1
6	1							
105	s2		104	[1] variabel	[6] record	2	1	1
59	1							
106	testedgecases		102	[3] prosedur	[0] undef	15	1	0
0	1							
107	dummy		-1	[1] variabel	[1] int	-1	1	1
5	1							
108	minval		107	[1] variabel	[1] int	-1	1	1
6	1							
109	maxval		108	[1] variabel	[1] int	-1	1	1
7	1							
110	epsilon		109	[1] variabel	[2] real	-1	1	1
8	1							
=====								
BTAB (Block Table)								
=====								
blocks	last	lpar	psze	vsze				

0	28	0	0	0				
1	106	-1	0	1853				
2	39	0	0	53				
3	43	0	0	1592				
4	62	61	3	1				
5	67	65	101	2				
6	71	70	2	53				
7	75	75	3	0				
8	77	-1	0	1				
9	82	79	1	3				
10	87	86	3	1				
11	92	91	3	1				
12	97	94	1	3				
13	101	99	1	2				
14	105	103	1	106				
15	110	107	1	3				
=====								
==								
ATAB (Array Table)								
=====								
==								
arrays	xtyp	etyp	eref	low	high	elsz	size	

--								
0	[1] int	[1] int	-1	1	10	1	10	
1	[1] int	[5] array	0	1	10	10	100	
2	[1] int	[4] char	-1	1	50	1	50	
3	[1] int	[6] record	2	1	30	53	1590	

```
4          [1] int  [1] int  -1      0      99      1      100
```

```
=====
```

```
===== Decorated AST Output =====
```

```
Program: ComprehensiveTest
```

```
Declarations:
```

```
  ConstDecl: MAXSIZE  -> index:30, type:integer, lev:0
```

```
    Value:
```

```
      Number: 100  -> type:integer
```

```
  ConstDecl: PIVALUE  -> index:31, type:real, lev:0
```

```
    Value:
```

```
      Number: 3.14159  -> type:real
```

```
  ConstDecl: APPNAME  -> index:32, type:string, lev:0
```

```
    Value:
```

```
      String: 'SemanticTester'  -> type:string
```

```
  ConstDecl: NEWLINE  -> index:33, type:string, lev:0
```

```
    Value:
```

```
      String: '\n'  -> type:string
```

```
  TypeDecl: IndexRange  -> index:34, type:integer, lev:0
```

```
    Definition:
```

```
      BuiltinType: integer  -> type:integer
```

```
  TypeDecl: Matrix  -> index:35, type:larik, lev:0
```

```
    Definition:
```

```
      ArrayType: [1..10]  -> index:1, type:larik
```

```
        ElementType:
```

```
          ArrayType: [1..10]  -> index:0, type:larik
```

```
            ElementType:
```

```
              BuiltinType: integer  -> type:integer
```

```
  TypeDecl: Student  -> index:40, type:rekaman, lev:0
```

```
    Definition:
```

```
      RecordType:  -> index:2, type:rekaman
```

```
        id:
```

```
          BuiltinType: integer  -> type:integer
```

```
        name:
```

```
          ArrayType: [1..50]  -> index:2, type:larik
```

```
            ElementType:
```

```
              BuiltinType: char  -> type:char
```

```
        gpa:
```

```
          BuiltinType: real  -> type:real
```

```
        active:
```

```
          BuiltinType: boolean  -> type:boolean
```

```
  TypeDecl: Classroom  -> index:44, type:rekaman, lev:0
```

```
    Definition:
```

```
      RecordType:  -> index:3, type:rekaman
```

```
        roomnumber:
```

```
          BuiltinType: integer  -> type:integer
```

```
        students:
```

```
          ArrayType: [1..30]  -> index:3, type:larik
```

```
            ElementType:
```

```
              CustomType: Student  -> index:40, type:rekaman, lev:0
```

```
        capacity:
```

```
          BuiltinType: integer  -> type:integer
```

```
  TypeDecl: IntArray  -> index:45, type:larik, lev:0
```

```
    Definition:
```

```
      ArrayType: [0..99]  -> index:4, type:larik
```

```

    ElementType:
      BuiltinType: integer -> type:integer
VarDecl: i -> index:46, type:integer, lev:0
  Type:
    BuiltinType: integer -> type:integer
VarDecl: j -> index:47, type:integer, lev:0
  Type:
    BuiltinType: integer -> type:integer
VarDecl: k -> index:48, type:integer, lev:0
  Type:
    BuiltinType: integer -> type:integer
VarDecl: x -> index:49, type:real, lev:0
  Type:
    BuiltinType: real -> type:real
VarDecl: y -> index:50, type:real, lev:0
  Type:
    BuiltinType: real -> type:real
VarDecl: z -> index:51, type:real, lev:0
  Type:
    BuiltinType: real -> type:real
VarDecl: flag -> index:52, type:boolean, lev:0
  Type:
    BuiltinType: boolean -> type:boolean
VarDecl: done -> index:53, type:boolean, lev:0
  Type:
    BuiltinType: boolean -> type:boolean
VarDecl: grid -> index:54, type:larik, lev:0
  Type:
    CustomType: Matrix -> index:35, type:larik, lev:0
VarDecl: class -> index:55, type:rekaman, lev:0
  Type:
    CustomType: Classroom -> index:44, type:rekaman, lev:0
VarDecl: scores -> index:56, type:larik, lev:0
  Type:
    CustomType: IntArray -> index:45, type:larik, lev:0
VarDecl: tempstudent -> index:57, type:rekaman, lev:0
  Type:
    CustomType: Student -> index:40, type:rekaman, lev:0
Function: calculate -> index:58, type:boolean, lev:0
Parameters:
  a:
    BuiltinType: integer -> type:integer
  b:
    BuiltinType: real -> type:real
  result:
    BuiltinType: real -> type:real
Return Type:
  BuiltinType: boolean -> type:boolean
Declarations:
  VarDecl: temp -> index:62, type:real, lev:1
  Type:
    BuiltinType: real -> type:real
Body:
  Block: -> index:4, lev:1
    Assignment: -> type:void
    Target:

```

```

    Variable: temp -> index:62, type:real, lev:1
Value:
    BinOp: * -> type:real
    Left:
        Variable: a -> index:59, type:integer
    Right:
        Variable: b -> index:60, type:real
Assignment: -> type:void
Target:
    Variable: result -> index:61, type:real, lev:1
Value:
    BinOp: + -> type:real
    Left:
        Variable: temp -> index:62, type:real
    Right:
        Variable: PIVALUE -> index:31, type:real
If: -> lev:1
Condition:
    BinOp: > -> type:boolean
    Left:
        Variable: result -> index:61, type:real
    Right:
        Number: 100.0 -> type:real
Then:
    Assignment: -> type:void
    Target:
        Variable: calculate -> index:58, type:boolean, lev:0
Value:
    Boolean: True -> type:boolean
Else:
    Assignment: -> type:void
    Target:
        Variable: calculate -> index:58, type:boolean, lev:0
Value:
    Boolean: False -> type:boolean
Function: sumarray -> index:63, type:integer, lev:0
Parameters:
    arr:
        CustomType: IntArray -> index:45, type:larik, lev:0
    size:
        BuiltinType: integer -> type:integer
Return Type:
    BuiltinType: integer -> type:integer
Declarations:
    VarDecl: total -> index:66, type:integer, lev:1
    Type:
        BuiltinType: integer -> type:integer
    VarDecl: idx -> index:67, type:integer, lev:1
    Type:
        BuiltinType: integer -> type:integer
Body:
    Block: -> index:5, lev:1
    Assignment: -> type:void
    Target:
        Variable: total -> index:66, type:integer, lev:1
Value:

```

```

    Number: 0 -> type:integer
For: idx -> lev:1
  Start:
    Number: 0 -> type:integer
    Direction: ke
  End:
    BinOp: - -> type:integer
    Left:
      Variable: size -> index:65, type:integer
    Right:
      Number: 1 -> type:integer
  Body:
    Assignment: -> type:void
    Target:
      Variable: total -> index:66, type:integer, lev:1
    Value:
      BinOp: + -> type:integer
      Left:
        Variable: total -> index:66, type:integer
      Right:
        ArrayAccess: -> index:4
        Variable:
          Variable: arr -> index:64, type:larik
        Index:
          Variable: idx -> index:67, type:integer
    Assignment: -> type:void
    Target:
      Variable: sumarray -> index:63, type:integer, lev:0
    Value:
      Variable: total -> index:66, type:integer
Procedure: swapstudents -> index:68, lev:0
Parameters:
  s1:
    CustomType: Student -> index:40, type:rekaman, lev:0
  s2:
    CustomType: Student -> index:40, type:rekaman, lev:0
Declarations:
  VarDecl: temp -> index:71, type:rekaman, lev:1
  Type:
    CustomType: Student -> index:40, type:rekaman, lev:0
Body:
  Block: -> index:6, lev:1
  Assignment: -> type:void
  Target:
    Variable: temp -> index:71, type:rekaman, lev:1
  Value:
    Variable: s1 -> index:69, type:rekaman
  Assignment: -> type:void
  Target:
    Variable: s1 -> index:69, type:rekaman, lev:1
  Value:
    Variable: s2 -> index:70, type:rekaman
  Assignment: -> type:void
  Target:
    Variable: s2 -> index:70, type:rekaman, lev:1
  Value:

```

```

        Variable: temp -> index:71, type:rekaman
Procedure: updateclassroom -> index:72, lev:0
Parameters:
  c:
    CustomType: Classroom -> index:44, type:rekaman, lev:0
  room:
    BuiltinType: integer -> type:integer
  cap:
    BuiltinType: integer -> type:integer
Declarations: (none)
Body:
  Block: -> index:7, lev:1
    Assignment: -> type:void
    Target:
      RecordAccess: -> index:3, lev:41
      Variable:
        Variable: c -> index:73, type:rekaman
        Field: roomnumber
    Value:
      Variable: room -> index:74, type:integer
    Assignment: -> type:void
    Target:
      RecordAccess: -> index:3, lev:43
      Variable:
        Variable: c -> index:73, type:rekaman
        Field: capacity
    Value:
      Variable: cap -> index:75, type:integer
Procedure: testcomplexaccess -> index:76, lev:0
Parameters: (none)
Declarations:
  VarDecl: idx -> index:77, type:integer, lev:1
  Type:
    BuiltinType: integer -> type:integer
Body:
  Block: -> index:8, lev:1
    Assignment: -> type:void
    Target:
      RecordAccess: -> index:2, lev:36
      Variable:
        ArrayAccess: -> index:3
        Variable:
          RecordAccess: -> index:3, lev:42
          Variable:
            Variable: class -> index:55, type:rekaman
            Field: students
          Index:
            Number: 1 -> type:integer
        Field: id
    Value:
      Number: 1001 -> type:integer
    Assignment: -> type:void
    Target:
      RecordAccess: -> index:2, lev:38
      Variable:
        ArrayAccess: -> index:3

```

```

    Variable:
      RecordAccess: -> index:3, lev:42
      Variable:
        Variable: class -> index:55, type:rekaman
        Field: students
      Index:
        Number: 1 -> type:integer
    Field: gpa
  Value:
    Number: 3.75 -> type:real
Assignment: -> type:void
Target:
  RecordAccess: -> index:2, lev:39
  Variable:
    ArrayAccess: -> index:3
    Variable:
      RecordAccess: -> index:3, lev:42
      Variable:
        Variable: class -> index:55, type:rekaman
        Field: students
      Index:
        Number: 1 -> type:integer
    Field: active
  Value:
    Boolean: True -> type:boolean
Assignment: -> type:void
Target:
  Variable: tempstudent -> index:57, type:rekaman, lev:0
Value:
  ArrayAccess: -> index:3
  Variable:
    RecordAccess: -> index:3, lev:42
    Variable:
      Variable: class -> index:55, type:rekaman
      Field: students
    Index:
      Number: 1 -> type:integer
Assignment: -> type:void
Target:
  ArrayAccess: -> index:2
  Variable:
    RecordAccess: -> index:2, lev:37
    Variable:
      ArrayAccess: -> index:3
      Variable:
        RecordAccess: -> index:3, lev:42
        Variable:
          Variable: class -> index:55, type:rekaman
          Field: students
        Index:
          Number: 1 -> type:integer
      Field: name
    Index:
      Number: 1 -> type:integer
Value:
  Char: 'J' -> type:char

```



```

Assignment: -> type:void
Target:
  ArrayAccess: -> index:2
  Variable:
    RecordAccess: -> index:2, lev:37
    Variable:
      ArrayAccess: -> index:3
      Variable:
        RecordAccess: -> index:3, lev:42
        Variable:
          Variable: class -> index:55, type:rekaman
          Field: students
        Index:
          Number: 1 -> type:integer
      Field: name
    Index:
      Number: 2 -> type:integer
Value:
  Char: 'o' -> type:char
Assignment: -> type:void
Target:
  ArrayAccess: -> index:2
  Variable:
    RecordAccess: -> index:2, lev:37
    Variable:
      ArrayAccess: -> index:3
      Variable:
        RecordAccess: -> index:3, lev:42
        Variable:
          Variable: class -> index:55, type:rekaman
          Field: students
        Index:
          Number: 1 -> type:integer
      Field: name
    Index:
      Number: 3 -> type:integer
Value:
  Char: 'h' -> type:char
Assignment: -> type:void
Target:
  ArrayAccess: -> index:2
  Variable:
    RecordAccess: -> index:2, lev:37
    Variable:
      ArrayAccess: -> index:3
      Variable:
        RecordAccess: -> index:3, lev:42
        Variable:
          Variable: class -> index:55, type:rekaman
          Field: students
        Index:
          Number: 1 -> type:integer
      Field: name
    Index:
      Number: 4 -> type:integer
Value:

```

```

Char: 'n' -> type:char
Procedure: testcontrolflow -> index:78, lev:0
Parameters:
  n:
    BuiltinType: integer -> type:integer
Declarations:
  VarDecl: counter -> index:80, type:integer, lev:1
    Type:
      BuiltinType: integer -> type:integer
  VarDecl: value -> index:81, type:integer, lev:1
    Type:
      BuiltinType: integer -> type:integer
  VarDecl: condition -> index:82, type:boolean, lev:1
    Type:
      BuiltinType: boolean -> type:boolean
Body:
  Block: -> index:9, lev:1
    If: -> lev:1
      Condition:
        BinOp: > -> type:boolean
        Left:
          Variable: n -> index:79, type:integer
        Right:
          Number: 0 -> type:integer
      Then:
        Block: -> index:9, lev:1
          Assignment: -> type:void
          Target:
            Variable: value -> index:81, type:integer, lev:1
          Value:
            BinOp: * -> type:integer
            Left:
              Variable: n -> index:79, type:integer
            Right:
              Number: 2 -> type:integer
        Else:
          Block: -> index:9, lev:1
            Assignment: -> type:void
            Target:
              Variable: value -> index:81, type:integer, lev:1
            Value:
              Number: 0 -> type:integer
      Assignment: -> type:void
      Target:
        Variable: counter -> index:80, type:integer, lev:1
      Value:
        Number: 0 -> type:integer
    While: -> lev:1
      Condition:
        BinOp: < -> type:boolean
        Left:
          Variable: counter -> index:80, type:integer
        Right:
          Variable: n -> index:79, type:integer
      Body:
        Block: -> index:9, lev:1

```

```

        Assignment: -> type:void
        Target:
            Variable: counter -> index:80, type:integer, lev:1
        Value:
            BinOp: + -> type:integer
            Left:
                Variable: counter -> index:80, type:integer
            Right:
                Number: 1 -> type:integer
For: i -> lev:1
    Start:
        Number: 1 -> type:integer
    Direction: ke
    End:
        Number: 10 -> type:integer
    Body:
        Block: -> index:9, lev:1
        Assignment: -> type:void
        Target:
            ArrayAccess: -> index:4
            Variable:
                Variable: scores -> index:56, type:larik
            Index:
                Variable: i -> index:46, type:integer
        Value:
            BinOp: * -> type:integer
            Left:
                Variable: i -> index:46, type:integer
            Right:
                Variable: i -> index:46, type:integer
For: i -> lev:1
    Start:
        Number: 10 -> type:integer
    Direction: turunke
    End:
        Number: 1 -> type:integer
    Body:
        Block: -> index:9, lev:1
        Assignment: -> type:void
        Target:
            Variable: value -> index:81, type:integer, lev:1
        Value:
            ArrayAccess: -> index:4
            Variable:
                Variable: scores -> index:56, type:larik
            Index:
                Variable: i -> index:46, type:integer
Assignment: -> type:void
    Target:
        Variable: counter -> index:80, type:integer, lev:1
    Value:
        Number: 0 -> type:integer
Repeat: -> lev:1
    Statements:
        Assignment: -> type:void
        Target:

```

```

        Variable: counter -> index:80, type:integer, lev:1
Value:
    BinOp: + -> type:integer
        Left:
            Variable: counter -> index:80, type:integer
        Right:
            Number: 1 -> type:integer
Until:
    BinOp: >= -> type:boolean
        Left:
            Variable: counter -> index:80, type:integer
        Right:
            Variable: n -> index:79, type:integer
Case: -> lev:1
    Selector:
        Variable: n -> index:79, type:integer
    Branches:
        Branch 1:
            Labels:
                Number: 1 -> type:integer
            Statement:
                Assignment: -> type:void
                Target:
                    Variable: value -> index:81, type:integer, lev:1
                Value:
                    Number: 10 -> type:integer
        Branch 2:
            Labels:
                Number: 2 -> type:integer
            Statement:
                Assignment: -> type:void
                Target:
                    Variable: value -> index:81, type:integer, lev:1
                Value:
                    Number: 20 -> type:integer
        Branch 3:
            Labels:
                Number: 5 -> type:integer
            Statement:
                Assignment: -> type:void
                Target:
                    Variable: value -> index:81, type:integer, lev:1
                Value:
                    Number: 50 -> type:integer
        Branch 4:
            Labels:
                Number: 10 -> type:integer
            Statement:
                Assignment: -> type:void
                Target:
                    Variable: value -> index:81, type:integer, lev:1
                Value:
                    Number: 100 -> type:integer
Function: complexexpression -> index:83, type:real, lev:0
Parameters:
    a:

```

```

    BuiltinType: integer -> type:integer
  b:
    BuiltinType: integer -> type:integer
  c:
    BuiltinType: integer -> type:integer
Return Type:
  BuiltinType: real -> type:real
Declarations:
  VarDecl: result -> index:87, type:real, lev:1
    Type:
      BuiltinType: real -> type:real
Body:
  Block: -> index:10, lev:1
    Assignment: -> type:void
      Target:
        Variable: result -> index:87, type:real, lev:1
      Value:
        BinOp: / -> type:real
          Left:
            BinOp: * -> type:integer
              Left:
                BinOp: + -> type:integer
                  Left:
                    Variable: a -> index:84, type:integer
                  Right:
                    Variable: b -> index:85, type:integer
                Right:
                  Variable: c -> index:86, type:integer
              Right:
                Number: 2.0 -> type:real
          Assignment: -> type:void
            Target:
              Variable: result -> index:87, type:real, lev:1
            Value:
              BinOp: + -> type:real
                Left:
                  Variable: result -> index:87, type:real
                Right:
                  BinOp: / -> type:real
                    Left:
                      BinOp: * -> type:integer
                        Left:
                          BinOp: - -> type:integer
                            Left:
                              Variable: a -> index:84, type:integer
                            Right:
                              Variable: b -> index:85, type:integer
                        Right:
                          BinOp: + -> type:integer
                            Left:
                              Variable: c -> index:86, type:integer
                            Right:
                              Number: 5 -> type:integer
                      Right:
                        Number: 3.0 -> type:real
                    Assignment: -> type:void

```



```

        Right:
            Variable: flag -> index:91, type:boolean
Assignment: -> type:void
Target:
    Variable: temp -> index:92, type:boolean, lev:1
Value:
    UnaryOp: tidak -> type:boolean
    Operand:
        Variable: temp -> index:92, type:boolean
Assignment: -> type:void
Target:
    Variable: testboolean -> index:88, type:boolean, lev:0
Value:
    BinOp: atau -> type:boolean
    Left:
        Variable: temp -> index:92, type:boolean
    Right:
        BinOp: dan -> type:boolean
        Left:
            BinOp: = -> type:boolean
            Left:
                Variable: x -> index:89, type:integer
            Right:
                Variable: y -> index:90, type:integer
        Right:
            BinOp: <> -> type:boolean
            Left:
                Variable: x -> index:89, type:integer
            Right:
                Number: 0 -> type:integer
Procedure: testnestedcalls -> index:93, lev:0
Parameters:
    dummy:
        BuiltinType: integer -> type:integer
Declarations:
    VarDecl: success -> index:95, type:boolean, lev:1
    Type:
        BuiltinType: boolean -> type:boolean
    VarDecl: total -> index:96, type:integer, lev:1
    Type:
        BuiltinType: integer -> type:integer
    VarDecl: res -> index:97, type:real, lev:1
    Type:
        BuiltinType: real -> type:real
Body:
    Block: -> index:12, lev:1
    Assignment: -> type:void
    Target:
        Variable: total -> index:96, type:integer, lev:1
    Value:
        FunctionCall: sumarray -> index:63, type:integer, lev:0
        Arguments:
            Variable: scores -> index:56, type:larik
            Number: 50 -> type:integer
    Assignment: -> type:void
    Target:

```

```

    Variable: success -> index:95, type:boolean, lev:1
Value:
    FunctionCall: calculate -> index:58, type:boolean, lev:0
    Arguments:
        Number: 10 -> type:integer
        Number: 2.5 -> type:real
        Variable: res -> index:97, type:real, lev:1
If: -> lev:1
    Condition:
        FunctionCall: calculate -> index:58, type:boolean, lev:0
        Arguments:
            Number: 5 -> type:integer
            Number: 3.0 -> type:real
            Variable: res -> index:97, type:real, lev:1
    Then:
        Assignment: -> type:void
        Target:
            Variable: total -> index:96, type:integer, lev:1
        Value:
            FunctionCall: sumarray -> index:63, type:integer, lev:0
            Arguments:
                Variable: scores -> index:56, type:larik
                Number: 10 -> type:integer
Procedure: testarrays -> index:98, lev:0
    Parameters:
        dummy:
            BuiltinType: integer -> type:integer
    Declarations:
        VarDecl: a -> index:100, type:integer, lev:1
        Type:
            BuiltinType: integer -> type:integer
        VarDecl: b -> index:101, type:integer, lev:1
        Type:
            BuiltinType: integer -> type:integer
    Body:
        Block: -> index:13, lev:1
        For: a -> lev:1
            Start:
                Number: 0 -> type:integer
            Direction: ke
            End:
                Number: 99 -> type:integer
            Body:
                Assignment: -> type:void
                Target:
                    ArrayAccess: -> index:4
                    Variable:
                        Variable: scores -> index:56, type:larik
                    Index:
                        Variable: a -> index:100, type:integer
                Value:
                    Variable: a -> index:100, type:integer
        For: a -> lev:1
            Start:
                Number: 1 -> type:integer
            Direction: ke

```



```

End:
  Number: 10  -> type:integer
Body:
  For: b  -> lev:1
    Start:
      Number: 1  -> type:integer
      Direction: ke
    End:
      Number: 10  -> type:integer
    Body:
      Assignment:  -> type:void
      Target:
        ArrayAccess:  -> index:0
        Variable:
          ArrayAccess:  -> index:1
          Variable:
            Variable: grid  -> index:54, type:larik
          Index:
            Variable: a  -> index:100, type:integer
          Index:
            Variable: b  -> index:101, type:integer
        Value:
          BinOp: *  -> type:integer
          Left:
            Variable: a  -> index:100, type:integer
          Right:
            Variable: b  -> index:101, type:integer
Procedure: testrecords  -> index:102, lev:0
Parameters:
  dummy:
    BuiltinType: integer  -> type:integer
Declarations:
  VarDecl: s1  -> index:104, type:rekaman, lev:1
  Type:
    CustomType: Student  -> index:40, type:rekaman, lev:0
  VarDecl: s2  -> index:105, type:rekaman, lev:1
  Type:
    CustomType: Student  -> index:40, type:rekaman, lev:0
Body:
  Block:  -> index:14, lev:1
    Assignment:  -> type:void
    Target:
      RecordAccess:  -> index:2, lev:36
      Variable:
        Variable: s1  -> index:104, type:rekaman
      Field: id
    Value:
      Number: 100  -> type:integer
    Assignment:  -> type:void
    Target:
      RecordAccess:  -> index:2, lev:38
      Variable:
        Variable: s1  -> index:104, type:rekaman
      Field: gpa
    Value:
      Number: 3.5  -> type:real

```

```

Assignment: -> type:void
Target:
  RecordAccess: -> index:2, lev:39
  Variable:
    Variable: s1 -> index:104, type:rekaman
    Field: active
  Value:
    Boolean: True -> type:boolean
Assignment: -> type:void
Target:
  Variable: s2 -> index:105, type:rekaman, lev:1
Value:
  Variable: s1 -> index:104, type:rekaman
Assignment: -> type:void
Target:
  RecordAccess: -> index:2, lev:36
  Variable:
    Variable: s2 -> index:105, type:rekaman
    Field: id
  Value:
    Number: 200 -> type:integer
ProcedureCall: swapstudents -> index:68, lev:0
Arguments:
  Variable: s1 -> index:104, type:rekaman, lev:1
  Variable: s2 -> index:105, type:rekaman, lev:1
Procedure: testedgecases -> index:106, lev:0
Parameters:
  dummy:
    BuiltinType: integer -> type:integer
Declarations:
  VarDecl: minval -> index:108, type:integer, lev:1
  Type:
    BuiltinType: integer -> type:integer
  VarDecl: maxval -> index:109, type:integer, lev:1
  Type:
    BuiltinType: integer -> type:integer
  VarDecl: epsilon -> index:110, type:real, lev:1
  Type:
    BuiltinType: real -> type:real
Body:
  Block: -> index:15, lev:1
  Assignment: -> type:void
  Target:
    Variable: minval -> index:108, type:integer, lev:1
  Value:
    Number: 0 -> type:integer
  Assignment: -> type:void
  Target:
    Variable: maxval -> index:109, type:integer, lev:1
  Value:
    Variable: MAXSIZE -> index:30, type:integer
  Assignment: -> type:void
  Target:
    Variable: epsilon -> index:110, type:real, lev:1
  Value:
    Number: 0.0001 -> type:real

```

```

Assignment: -> type:void
  Target:
    Variable: minval -> index:108, type:integer, lev:1
  Value:
    UnaryOp: - -> type:integer
    Operand:
      Number: 100 -> type:integer
Assignment: -> type:void
  Target:
    ArrayAccess: -> index:4
    Variable:
      Variable: scores -> index:56, type:larik
    Index:
      Number: 0 -> type:integer
  Value:
    Variable: minval -> index:108, type:integer
Assignment: -> type:void
  Target:
    ArrayAccess: -> index:4
    Variable:
      Variable: scores -> index:56, type:larik
    Index:
      Number: 99 -> type:integer
  Value:
    Variable: maxval -> index:109, type:integer
Assignment: -> type:void
  Target:
    ArrayAccess: -> index:0
    Variable:
      ArrayAccess: -> index:1
      Variable:
        Variable: grid -> index:54, type:larik
      Index:
        Number: 1 -> type:integer
    Index:
      Number: 1 -> type:integer
  Value:
    Number: 1 -> type:integer
Assignment: -> type:void
  Target:
    ArrayAccess: -> index:0
    Variable:
      ArrayAccess: -> index:1
      Variable:
        Variable: grid -> index:54, type:larik
      Index:
        Number: 10 -> type:integer
    Index:
      Number: 10 -> type:integer
  Value:
    Number: 100 -> type:integer
Body:
  Block: -> index:1, lev:0
    Assignment: -> type:void
      Target:
        Variable: i -> index:46, type:integer, lev:0

```

```

    Value:
      Number: 0 -> type:integer
Assignment: -> type:void
Target:
  Variable: x -> index:49, type:real, lev:0
Value:
  Number: 0.0 -> type:real
Assignment: -> type:void
Target:
  Variable: flag -> index:52, type:boolean, lev:0
Value:
  Boolean: False -> type:boolean
Assignment: -> type:void
Target:
  Variable: i -> index:46, type:integer, lev:0
Value:
  Variable: MAXSIZE -> index:30, type:integer
Assignment: -> type:void
Target:
  Variable: x -> index:49, type:real, lev:0
Value:
  Variable: PIVALUE -> index:31, type:real
ProcedureCall: testcomplexaccess -> index:76, lev:0
(no arguments)
ProcedureCall: testcontrolflow -> index:78, lev:0
Arguments:
  Number: 10 -> type:integer
ProcedureCall: testnestedcalls -> index:93, lev:0
Arguments:
  Variable: i -> index:46, type:integer
ProcedureCall: testarrays -> index:98, lev:0
Arguments:
  Variable: i -> index:46, type:integer
ProcedureCall: testrecords -> index:102, lev:0
Arguments:
  Variable: i -> index:46, type:integer
ProcedureCall: testedgecases -> index:106, lev:0
Arguments:
  Variable: i -> index:46, type:integer
Assignment: -> type:void
Target:
  Variable: x -> index:49, type:real, lev:0
Value:
  FunctionCall: complexexpression -> index:83, type:real, lev:0
    Arguments:
      Number: 5 -> type:integer
      Number: 3 -> type:integer
      Number: 2 -> type:integer
Assignment: -> type:void
Target:
  Variable: flag -> index:52, type:boolean, lev:0
Value:
  FunctionCall: testboolean -> index:88, type:boolean, lev:0
    Arguments:
      Number: 10 -> type:integer
      Number: 20 -> type:integer

```

```

        Boolean: True -> type:boolean
Assignment: -> type:void
Target:
    Variable: i -> index:46, type:integer, lev:0
Value:
    FunctionCall: sumarray -> index:63, type:integer, lev:0
    Arguments:
        Variable: scores -> index:56, type:larik
        Number: 100 -> type:integer
If: -> lev:0
Condition:
    FunctionCall: calculate -> index:58, type:boolean, lev:0
    Arguments:
        Variable: i -> index:46, type:integer
        Variable: x -> index:49, type:real
        Variable: y -> index:50, type:real, lev:0
Then:
    Block: -> index:1, lev:0
    ProcedureCall: updateclassroom -> index:72, lev:0
    Arguments:
        Variable: class -> index:55, type:rekaman, lev:0
        Number: 101 -> type:integer
        Number: 30 -> type:integer

```

Kasus 9

Input file: testadr.pas

```

program BerhasilAmin;

konstanta
    NAMAKURSUS = 'TBFO IF2224';
    KARAKTERA = 'a';
    MAXSIZE = -100;
    PI = 3.14159;

tipe
    Angka5 = larik[1 .. 5] dari integer;
    recordtipe = rekaman
        id: integer;
        nilai: real;
    selesai;
    M = larik [1..10] dari larik [1..10] dari integer;

variabel
    tabel: M;
    global1Var: integer;
    a: real;
    varRecord: recordtipe;
    global2Var: integer;
    writeln: integer;

fungsi f(param: integer): integer;

```

```

mulai
    f := 6 + param;
selesai;

prosedur hitung(param: integer);
variabel
    a: char;
    lokal1Var: integer;
    lokalArr : Angka5;
    lokal2Var: integer;
mulai
    a := 'b';
    lokalArr[1] := 2;
    lokal1Var := 50;
    lokal2Var := 100 + lokal1Var;
    global1Var := param + lokal2Var;
selesai;

mulai
    tabel[1][2] := 5;
    varRecord.id := 5;
    varRecord.nilai := 1.8;

    global1Var := 10 + MAXSIZE;
    hitung(global1Var);
    global2Var := 2 * global1Var;
    global1Var := global1Var + varRecord.id;
    a := 5.2 + varRecord.nilai;
    global2Var := global2Var + f(global1Var);

    jika false maka
        writeln := global2Var
    selainitu
        writeln := 2;
selesai.

```

Output

```

Program: BerhasilAmin
Declarations:
  ConstDecl: NAMA KURSUS
    Value:
      String: 'TBFO IF2224'
  ConstDecl: KARAKTERA
    Value:
      Char: 'a'
  ConstDecl: MAXSIZE
    Value:
      Number: -100
  ConstDecl: PI
    Value:
      Number: 3.14159
  TypeDecl: Angka5
    Definition:
      ArrayType: [1..5]

```

```

        ElementType:
            BuiltinType: integer
TypeDecl: recordtipe
Definition:
    RecordType:
        id:
            BuiltinType: integer
        nilai:
            BuiltinType: real
TypeDecl: M
Definition:
    ArrayType: [1..10]
    ElementType:
        ArrayType: [1..10]
        ElementType:
            BuiltinType: integer
VarDecl: tabel
Type:
    CustomType: M
VarDecl: global1Var
Type:
    BuiltinType: integer
VarDecl: a
Type:
    BuiltinType: real
VarDecl: varRecord
Type:
    CustomType: recordtipe
VarDecl: global2Var
Type:
    BuiltinType: integer
VarDecl: writeln
Type:
    BuiltinType: integer
Function: f
Parameters:
    param:
        BuiltinType: integer
Return Type:
    BuiltinType: integer
Declarations: (none)
Body:
    Block:
        Assignment:
            Target:
                Variable: f
            Value:
                BinOp: +
                Left:
                    Number: 6
                Right:
                    Variable: param
Procedure: hitung
Parameters:
    param:
        BuiltinType: integer

```

```
Declarations:
  VarDecl: a
    Type:
      BuiltinType: char
  VarDecl: lokal1Var
    Type:
      BuiltinType: integer
  VarDecl: lokalArr
    Type:
      CustomType: Angka5
  VarDecl: lokal2Var
    Type:
      BuiltinType: integer
Body:
  Block:
    Assignment:
      Target:
        Variable: a
      Value:
        Char: 'b'
    Assignment:
      Target:
        ArrayAccess:
          Variable:
            Variable: lokalArr
          Index:
            Number: 1
        Value:
          Number: 2
    Assignment:
      Target:
        Variable: lokal1Var
      Value:
        Number: 50
    Assignment:
      Target:
        Variable: lokal2Var
      Value:
        BinOp: +
        Left:
          Number: 100
        Right:
          Variable: lokal1Var
    Assignment:
      Target:
        Variable: global1Var
      Value:
        BinOp: +
        Left:
          Variable: param
        Right:
          Variable: lokal2Var
  Body:
    Block:
      Assignment:
        Target:
```



```

    ArrayAccess:
      Variable:
        ArrayAccess:
          Variable:
            Variable: tabel
          Index:
            Number: 1
        Index:
          Number: 2
    Value:
      Number: 5
Assignment:
  Target:
    RecordAccess:
      Variable:
        Variable: varRecord
      Field: id
    Value:
      Number: 5
Assignment:
  Target:
    RecordAccess:
      Variable:
        Variable: varRecord
      Field: nilai
    Value:
      Number: 1.8
Assignment:
  Target:
    Variable: global1Var
  Value:
    BinOp: +
    Left:
      Number: 10
    Right:
      Variable: MAXSIZE
ProcedureCall: hitung
  Arguments:
    Variable: global1Var
Assignment:
  Target:
    Variable: global2Var
  Value:
    BinOp: *
    Left:
      Number: 2
    Right:
      Variable: global1Var
Assignment:
  Target:
    Variable: global1Var
  Value:
    BinOp: +
    Left:
      Variable: global1Var
    Right:

```

```

        RecordAccess:
            Variable:
                Variable: varRecord
                Field: id
Assignment:
    Target:
        Variable: a
    Value:
        BinOp: +
        Left:
            Number: 5.2
        Right:
            RecordAccess:
                Variable:
                    Variable: varRecord
                    Field: nilai
Assignment:
    Target:
        Variable: global2Var
    Value:
        BinOp: +
        Left:
            Variable: global2Var
        Right:
            FunctionCall: f
            Arguments:
                Variable: global1Var
If:
    Condition:
        Boolean: False
    Then:
        Assignment:
            Target:
                Variable: writeln
            Value:
                Variable: global2Var
    Else:
        Assignment:
            Target:
                Variable: writeln
            Value:
                Number: 2
===== Semantic Analysis Output =====

```

=====								
=====								
TAB (Identifier Table)								
=====								
idx	identifiers	link	obj	type	ref	nrm	lev	
adr	init							

0	dan	-1	[6] keyword	[0] undef	-1	0	-1	
0	1							
1	larik	0	[6] keyword	[0] undef	-1	0	-1	

0	1						
2	mulai	1	[6] keyword	[0] undef	-1	0	-1
0	1						
3	kasus	2	[6] keyword	[0] undef	-1	0	-1
0	1						
4	konstanta	3	[6] keyword	[0] undef	-1	0	-1
0	1						
5	div	4	[6] keyword	[0] undef	-1	0	-1
0	1						
6	turunke	5	[6] keyword	[0] undef	-1	0	-1
0	1						
7	lakukan	6	[6] keyword	[0] undef	-1	0	-1
0	1						
8	selainitu	7	[6] keyword	[0] undef	-1	0	-1
0	1						
9	selesai	8	[6] keyword	[0] undef	-1	0	-1
0	1						
10	untuk	9	[6] keyword	[0] undef	-1	0	-1
0	1						
11	fungsi	10	[6] keyword	[0] undef	-1	0	-1
0	1						
12	jika	11	[6] keyword	[0] undef	-1	0	-1
0	1						
13	mod	12	[6] keyword	[0] undef	-1	0	-1
0	1						
14	tidak	13	[6] keyword	[0] undef	-1	0	-1
0	1						
15	dari	14	[6] keyword	[0] undef	-1	0	-1
0	1						
16	atau	15	[6] keyword	[0] undef	-1	0	-1
0	1						
17	prosedur	16	[6] keyword	[0] undef	-1	0	-1
0	1						
18	program	17	[6] keyword	[0] undef	-1	0	-1
0	1						
19	rekaman	18	[6] keyword	[0] undef	-1	0	-1
0	1						
20	ulangi	19	[6] keyword	[0] undef	-1	0	-1
0	1						
21	string	20	[6] keyword	[0] undef	-1	0	-1
0	1						
22	maka	21	[6] keyword	[0] undef	-1	0	-1
0	1						
23	ke	22	[6] keyword	[0] undef	-1	0	-1
0	1						
24	tipe	23	[6] keyword	[0] undef	-1	0	-1
0	1						
25	sampai	24	[6] keyword	[0] undef	-1	0	-1
0	1						
26	var	25	[6] keyword	[0] undef	-1	0	-1
0	1						
27	selama	26	[6] keyword	[0] undef	-1	0	-1
0	1						
28	packed	27	[6] keyword	[0] undef	-1	0	-1
0	1						
29	BerhasilAmin	-1	[5] program	[0] undef	-1	1	0

0	1						
30	NAMAKURSUS	29	[0] konstanta	[7] string	-1	1	0
11	1						
31	KARAKTERA	30	[0] konstanta	[4] char	-1	1	0
97	1						
32	MAXSIZE	31	[0] konstanta	[1] int	-1	1	0
4294967196	1						
33	PI	32	[0] konstanta	[2] real	-1	1	0
3	1						
34	Angka5	33	[2] tipe	[5] array	0	1	0
5	1						
35	id	-1	[1] variabel	[1] int	-1	1	1
0	1						
36	nilai	35	[1] variabel	[2] real	-1	1	1
1	1						
37	recordtipe	34	[2] tipe	[6] record	2	1	0
2	1						
38	M	37	[2] tipe	[5] array	2	1	0
100	1						
39	tabel	38	[1] variabel	[5] array	2	1	0
5	1						
40	global1Var	39	[1] variabel	[1] int	-1	1	0
105	1						
41	a	40	[1] variabel	[2] real	-1	1	0
106	1						
42	varRecord	41	[1] variabel	[6] record	2	1	0
107	1						
43	global2Var	42	[1] variabel	[1] int	-1	1	0
109	1						
44	writeln	43	[1] variabel	[1] int	-1	1	0
110	1						
45	f	44	[4] fungsi	[1] int	3	1	0
0	1						
46	param	-1	[1] variabel	[1] int	-1	1	1
5	1						
47	hitung	45	[3] prosedur	[0] undef	4	1	0
0	1						
48	param	-1	[1] variabel	[1] int	-1	1	1
5	1						
49	a	48	[1] variabel	[4] char	-1	1	1
6	1						
50	lokal1Var	49	[1] variabel	[1] int	-1	1	1
7	1						
51	lokalArr	50	[1] variabel	[5] array	0	1	1
8	1						
52	lokal2Var	51	[1] variabel	[1] int	-1	1	1
13	1						

=====

BTAB (Block Table)

=====

blocks	last	lpar	psze	vsze
0	28	0	0	0
1	47	-1	0	106
2	36	0	0	2

3	46	46	1	0
4	52	48	1	8

```
=====
==
                                ATAB (Array Table)
=====
==
arrays   xtyp      etyp      eref    low    high    elsz    size
-----
--
0         [1] int   [1] int   -1      1      5      1      5
1         [1] int   [1] int   -1      1     10      1     10
2         [1] int   [5] array  1       1     10     10    100
=====
==
```

```
===== Decorated AST Output =====
Program: BerhasilAmin
Declarations:
  ConstDecl: NAMA KURSUS -> index:30, type:string, lev:0
    Value:
      String: 'TBFO IF2224' -> type:string
  ConstDecl: KARAKTERA -> index:31, type:char, lev:0
    Value:
      Char: 'a' -> type:char
  ConstDecl: MAXSIZE -> index:32, type:integer, lev:0
    Value:
      Number: -100 -> type:integer
  ConstDecl: PI -> index:33, type:real, lev:0
    Value:
      Number: 3.14159 -> type:real
  TypeDecl: Angka5 -> index:34, type:larik, lev:0
    Definition:
      ArrayType: [1..5] -> index:0, type:larik
      ElementType:
        BuiltinType: integer -> type:integer
  TypeDecl: recordtipe -> index:37, type:rekaman, lev:0
    Definition:
      RecordType: -> index:2, type:rekaman
      id:
        BuiltinType: integer -> type:integer
      nilai:
        BuiltinType: real -> type:real
  TypeDecl: M -> index:38, type:larik, lev:0
    Definition:
      ArrayType: [1..10] -> index:2, type:larik
      ElementType:
        ArrayType: [1..10] -> index:1, type:larik
        ElementType:
          BuiltinType: integer -> type:integer
  VarDecl: tabel -> index:39, type:larik, lev:0
    Type:
      CustomType: M -> index:38, type:larik, lev:0
  VarDecl: globalVar -> index:40, type:integer, lev:0
    Type:
```

```

    BuiltinType: integer -> type:integer
VarDecl: a -> index:41, type:real, lev:0
Type:
    BuiltinType: real -> type:real
VarDecl: varRecord -> index:42, type:rekaman, lev:0
Type:
    CustomType: recordtype -> index:37, type:rekaman, lev:0
VarDecl: global2Var -> index:43, type:integer, lev:0
Type:
    BuiltinType: integer -> type:integer
VarDecl: writeln -> index:44, type:integer, lev:0
Type:
    BuiltinType: integer -> type:integer
Function: f -> index:45, type:integer, lev:0
Parameters:
    param:
        BuiltinType: integer -> type:integer
Return Type:
    BuiltinType: integer -> type:integer
Declarations: (none)
Body:
    Block: -> index:3, lev:1
        Assignment: -> type:void
            Target:
                Variable: f -> index:45, type:integer, lev:0
            Value:
                BinOp: + -> type:integer
                    Left:
                        Number: 6 -> type:integer
                    Right:
                        Variable: param -> index:46, type:integer
Procedure: hitung -> index:47, lev:0
Parameters:
    param:
        BuiltinType: integer -> type:integer
Declarations:
    VarDecl: a -> index:49, type:char, lev:1
        Type:
            BuiltinType: char -> type:char
    VarDecl: lokal1Var -> index:50, type:integer, lev:1
        Type:
            BuiltinType: integer -> type:integer
    VarDecl: lokalArr -> index:51, type:larik, lev:1
        Type:
            CustomType: Angka5 -> index:34, type:larik, lev:0
    VarDecl: lokal2Var -> index:52, type:integer, lev:1
        Type:
            BuiltinType: integer -> type:integer
Body:
    Block: -> index:4, lev:1
        Assignment: -> type:void
            Target:
                Variable: a -> index:49, type:char, lev:1
            Value:
                Char: 'b' -> type:char
        Assignment: -> type:void

```

```

    Target:
      ArrayAccess: -> index:0
      Variable:
        Variable: lokalArr -> index:51, type:larik
      Index:
        Number: 1 -> type:integer
    Value:
      Number: 2 -> type:integer
  Assignment: -> type:void
  Target:
    Variable: lokal1Var -> index:50, type:integer, lev:1
  Value:
    Number: 50 -> type:integer
  Assignment: -> type:void
  Target:
    Variable: lokal2Var -> index:52, type:integer, lev:1
  Value:
    BinOp: + -> type:integer
    Left:
      Number: 100 -> type:integer
    Right:
      Variable: lokal1Var -> index:50, type:integer
  Assignment: -> type:void
  Target:
    Variable: global1Var -> index:40, type:integer, lev:0
  Value:
    BinOp: + -> type:integer
    Left:
      Variable: param -> index:48, type:integer
    Right:
      Variable: lokal2Var -> index:52, type:integer
Body:
  Block: -> index:1, lev:0
  Assignment: -> type:void
  Target:
    ArrayAccess: -> index:1
    Variable:
      ArrayAccess: -> index:2
      Variable:
        Variable: tabel -> index:39, type:larik
      Index:
        Number: 1 -> type:integer
    Index:
      Number: 2 -> type:integer
  Value:
    Number: 5 -> type:integer
  Assignment: -> type:void
  Target:
    RecordAccess: -> index:2, lev:35
    Variable:
      Variable: varRecord -> index:42, type:rekaman
    Field: id
  Value:
    Number: 5 -> type:integer
  Assignment: -> type:void
  Target:

```

```

    RecordAccess: -> index:2, lev:36
    Variable:
        Variable: varRecord -> index:42, type:rekaman
        Field: nilai
    Value:
        Number: 1.8 -> type:real
Assignment: -> type:void
    Target:
        Variable: global1Var -> index:40, type:integer, lev:0
    Value:
        BinOp: + -> type:integer
        Left:
            Number: 10 -> type:integer
        Right:
            Variable: MAXSIZE -> index:32, type:integer
ProcedureCall: hitung -> index:47, lev:0
    Arguments:
        Variable: global1Var -> index:40, type:integer
Assignment: -> type:void
    Target:
        Variable: global2Var -> index:43, type:integer, lev:0
    Value:
        BinOp: * -> type:integer
        Left:
            Number: 2 -> type:integer
        Right:
            Variable: global1Var -> index:40, type:integer
Assignment: -> type:void
    Target:
        Variable: global1Var -> index:40, type:integer, lev:0
    Value:
        BinOp: + -> type:integer
        Left:
            Variable: global1Var -> index:40, type:integer
        Right:
            RecordAccess: -> index:2, lev:35
            Variable:
                Variable: varRecord -> index:42, type:rekaman
                Field: id
Assignment: -> type:void
    Target:
        Variable: a -> index:41, type:real, lev:0
    Value:
        BinOp: + -> type:real
        Left:
            Number: 5.2 -> type:real
        Right:
            RecordAccess: -> index:2, lev:36
            Variable:
                Variable: varRecord -> index:42, type:rekaman
                Field: nilai
Assignment: -> type:void
    Target:
        Variable: global2Var -> index:43, type:integer, lev:0
    Value:
        BinOp: + -> type:integer

```



```

    Left:
      Variable: global2Var -> index:43, type:integer
    Right:
      FunctionCall: f -> index:45, type:integer, lev:0
      Arguments:
        Variable: global1Var -> index:40, type:integer
If: -> lev:0
  Condition:
    Boolean: False -> type:boolean
  Then:
    Assignment: -> type:void
    Target:
      Variable: writeln -> index:44, type:integer, lev:0
    Value:
      Variable: global2Var -> index:43, type:integer
  Else:
    Assignment: -> type:void
    Target:
      Variable: writeln -> index:44, type:integer, lev:0
    Value:
      Number: 2 -> type:integer

```

KESIMPULAN DAN SARAN

Pada milestone ini telah berhasil diimplementasikan tahap semantic analysis untuk Pascal-S Compiler. Sistem mampu membangun AST dari parse tree, melakukan pengecekan tipe, validasi deklarasi identifiier, serta pengelolaan scope menggunakan symbol table (tab, btab, dan atab). Melalui mekanisme visitor, setiap node AST dianalisis secara terstruktur sehingga program dapat diverifikasi maknanya sesuai aturan semantik Pascal-S.

Ke depan, implementasi dapat ditingkatkan dengan menambahkan deteksi error yang lebih komprehensif dan error reporting yang lebih informatif agar memudahkan debugging. Selain itu, optimalisasi struktur data symbol table serta perluasan dukungan tipe komposit dan operasi kompleks dapat dilakukan untuk meningkatkan ketahanan dan fleksibilitas compiler pada kasus program yang lebih besar.

LAMPIRAN

- Link Release Repository Github
- Pembagian Tugas:

No.	NIM	Persentase Kontribusi
1.	13523023	25%
2.	13523025	25%

3.	13523051	25%
4.	13523111	25%

REFERENSI

- Aho, Alfred V. *Compilers: Principles, Techniques, & Tools*. Pearson/Addison Wesley, 2007,
[https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20\(2006\).pdf](https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20(2006).pdf). Accessed 16 October 2025.
- Hopcroft, John E., et al. *Introduction to Automata Theory, Languages, and Computation*. 3rd ed., Pearson/Addison Wesley, 2007,
https://cdn-edunex.itb.ac.id/29161-Formal-Language-Theory-and-Automata/1629640939613_Introduction-to-Automata-Theory,-Languages,-and-Computation-Edition-3.pdf. Accessed 16 October 2025.
- Wirth, Niklaus. *PASCAL-S. A Subset and its Implementation*.
<http://pascal.hansotten.com/uploads/pascals/PASCAL-S%20A%20subset%20and%20its%20Implementation%20012.pdf>. Accessed 16 October 2025.